SITE AUTOMATION

# VM Deploy Guide

VMs from a ServiceNow ticket with AAP: set it up step by step, how a run works, what to do when it goes wrong, every setting

Site Automation - for the deploy template's owners | site-automation 0.45.2 | October 2026

**What this is.** The VM deploy job, complete: the standard setup to start with; setting it up in AAP step by step - the execution environment, the vCenter role, the credentials, the job template and its survey, the approval workflow, the scheduled pickup, the server's service and its AAP inventory; how a run works; what requesters write; where each VM goes; configurations to copy; what to do when something goes wrong - and every setting with its default, an example and when to set it.

**How to read it**
New to it: chapter 1 (*Start here: the standard setup*), then chapter 2, *Set it up in AAP, step by step*, in order - each step says what you should see. A message to fix: *When something goes wrong*.
Rendered from `docs/VM_DEPLOY_GUIDE.md` and `roles/vmware_vm/defaults/main.yml` in the site-automation repository (version 0.45.2).

Host names and addresses in this guide are placeholders. Copy YAML from the repository files, not from the PDF.

# Contents

**1. What it does, and the standard setup** 4

What it does . . . 4

Start here: the standard setup . . . 5

**2. Set it up in AAP, step by step** 7

1. The execution environment . . . 7

2. The vCenter account and its role . . . 8

3. The templates in vCenter . . . 9

4. The credentials . . . 9

5. Find the names: the deploy settings finder . . . 10

6. The deploy job template . . . 10

7. The first runs: a dry run, then a real one . . . 11

8. The approval workflow: propose, approve, build . . . 11

9. The scheduled pickup . . . 12

10. The server's service: its notes, its AAP inventory . . . 14

11. Launch it from another tool (AAP's API) . . . 15

**3. How a run works** 17

The flow, step by step . . . 17

Report first, then build . . . 18

Which number to launch with . . . 19

The change: what is read, what must be true . . . 20

Closing the task: pick one . . . 22

Where the notes go . . . 23

Several VMs on one ticket . . . 24

Run it again: it picks up where it stopped . . . 25

**4. The request: what the ticket says** 26

What the requester writes . . . 26

The lines: key: value . . . 26

Plain words, without GenAI . . . 28

GenAI reads the ticket (optional) . . . 30

**5. Where the VM goes** 33

Who wins . . . 33

Leaving things out: the defaults . . . 33
Sites: site A goes to this cluster, site B to that one . . . 34
Name rules: the VM's name picks the cluster . . . 35
Automatic placement: the job picks the cluster and datastore . . . 35
Empty templates: network boot to MECM, or boot from a CD (kickstart) . . . 36

**6. Ready-made configurations 39**

**7. When something goes wrong 41**

The deploy job . . . 41
The scheduled pickup . . . 49

**8. Where the logic lives 50**

**9. Every VM deploy setting: its default, an example, when to set it 52**

Deploy a VM from a ServiceNow ticket (playbooks/vm_deploy_from_ticket.yml) . . . 52
The server's service, picked at launch: words in the VM's notes, an AAP inventory and group . . . 60
A request in plain words, without GenAI (off by default) . . . 61
GenAI reads a request written in plain words (a proof of concept, off by default) . . . 61
Pick up VM request tickets on a schedule (playbooks/vm_deploy_poll.yml) . . . 63

# 1. What it does, and the standard setup

The VM deploy job (`playbooks/vm_deploy_from_ticket.yml`) builds VMs in vCenter from a ServiceNow ticket. Someone asks for a server on a change (or an incident, a catalog task); AAP reads the ticket, checks the request against the rules you set, builds each VM from a template, and writes the result - with the VM's IP address - on the ticket. This guide is all of it in one place: setting it up in AAP step by step, how a run works, what requesters write, where each VM goes, configurations to copy, what to do when something goes wrong - and every setting with its default, an example and when to set it.

**Where your values go**
Every setting below goes in the deploy job template's **Variables** in AAP (or `playbooks/group_vars/all.yml`, which an update zip never overwrites). The names here are examples: `example.mil`, `SiteA`, `*AB01*`, `Server Team`, `rhel9-gold`. Put your own in AAP, never in the repository.

**How to read it:**

| You want to | Read |
|---|---|
| know what it does, in two minutes | *What it does*, next |
| set it up for the first time | *Start here: the standard setup*, then the chapter *Set it up in AAP, step by step* - in order, each step says what you should see |
| know why a run did what it did | *How a run works* |
| tell requesters what to write | *What the requester writes* |
| change where VMs go | *Where the VM goes* |
| copy a whole configuration | *Ready-made configurations* |
| fix a message | *When something goes wrong* |
| look up one setting | the last chapter of the PDF (*Every VM deploy setting*), or VARIABLES_REFERENCE.md |

## What it does

One run of the deploy job, launched with a ticket number:

1. **Reads** the ticket - and, for a change task, its change; for a change, its task.
2. **Checks** the request: the change approved and in Implement, your group's ticket, an allowed template, CPU, memory and disks within your caps, a free name in every vCenter. Anything wrong: a note on the ticket says what, and **nothing is built**.
3. **Places** each VM: its vCenter, cluster, datastore and folder - from the ticket, its site, its name, your defaults, or the least used place among the ones you allow.
4. **Builds** each VM: clone, CPU, memory, disks, guest customization (or a network boot to MECM), power on, its IP address.
5. **Writes** a note per VM on the ticket, and closes the task - only when you said so. The change is never closed.

A **dry run** (*Job type* **Check**) does 1 to 3, prints where every value came from and where each VM would go, and writes nothing. The **approval workflow** runs the job twice - propose, then build - with a person's approval between. A **schedule** can pick new tickets up and start that workflow by itself.

**Who does what:**

| Who | Does | Never does |
|---|---|---|
| **ServiceNow** | holds the request; shows the job's notes and questions | - |
| **AAP** (this job) | reads the ticket, checks everything, writes the notes, **builds the VMs in vCenter**, adds them to an AAP inventory (when the launch names a service), reports | builds anything your rules do not allow, or without a "Deploying" note on the ticket |
| **vCenter** | does what AAP asks: clone, size, customize, power on | - |
| **GenAI** (optional, off by default) | **copies the ticket text out as values** - names, the OS as written, CPU, memory, disks - quoting the words each comes from | builds or changes anything, sees a credential or vCenter, chooses a template or a place, decides whether to build |
| **The operator / approver** | launches the job or the workflow; reads the plan and approves (or denies) it in AAP | - |

**The templates in AAP** (each one is set up in the next chapter):

| Job template | Playbook | What it is for |
|---|---|---|
| VM - deploy from ServiceNow ticket | `playbooks/vm_deploy_from_ticket.yml` | **the deploy job**: launched with a ticket number; Check = a dry run |
| VM - deploy from ServiceNow ticket (workflow) | - (a workflow) | the deploy job as *propose*, a person's **approval**, then *build* - one launch |
| VM - pick up deploy tickets | `playbooks/vm_deploy_poll.yml` | optional: on a schedule, starts the workflow for each new VM request ticket |
| VM - deploy settings finder | `playbooks/vm_deploy_inventory.yml` | read-only: the names of your clusters, templates, datastores and folders, with lines to paste |

## Start here: the standard setup

Most sites need a handful of the settings. This is the setup to begin with - and to show:

- **Launched with the change** (`CHG...`); the change may be any group's, and must be approved and in Implement.
- **Work notes on the change and on its task** - the task found under the change by its words.
- **One switch** decides whether the job closes the task when every VM is built, or your team does.
- **The change is never closed.**

The standard setup

```
vm_deploy_templates: [rhel9-gold, WIN2022-TEMPLATE]       # only these are ever built
vm_deploy_template_specs: {"WIN2022*": Win2022-Sysprep}   # Windows: its Sysprep specification
vm_deploy_defaults: {cluster: PROD-CL01, datastore: DSC-PROD, folder: Servers/New}
vm_deploy_assignment_group: ""                    # the change may be any group's
vm_deploy_notes_also: [parent, tasks]             # every note on the change and on its task
vm_deploy_notes_tasks_query: "short_descriptionLIKEto deploy"   # which task: its words
vm_deploy_close_tasks: false                      # THE SWITCH: true = the task closed when built
vm_deploy_done_fields_by_table:                   # how a task is closed (only with the switch on)
  change_task: {state: "3", close_code: successful, close_notes: "VM deployed by AAP"}
vm_deploy_done_fields_issues_by_table:            # built, but e.g. no IP address in time
  change_task: {close_code: successful_issues}
report_email_to: vmteam@example.mil
```

- **The switch:** `vm_deploy_close_tasks: false` - every note goes on the task, your team closes it. `true` - the job closes it when every VM is built, with the `change_task` values. Nothing else changes.
- **The change stays open** because nothing gives it close values: never set `vm_deploy_done_fields` (it is for every table, the change too) and never add a `change_request` entry to `vm_deploy_done_fields_by_table`.
- **The task's words:** in ServiceNow, filter the change task list (*Short description contains* ...), right-click the filter and *Copy query*; paste it as `vm_deploy_notes_tasks_query`. The job asks for the change's open tasks that match - `short_descriptionLIKEto deploy^change_request=<the change>^active=true`. `LIKE` is *contains*, in any case. No task matches: the VMs are still built, the notes go on the change.
- **Close values** are your instance's **values**, not the labels (`3` and `successful` are ServiceNow's own): right-click the task's *State* (or *Close code*) field, *Show Choice List*, the *Value* column.
- **Launch with the change's number.** Launched with the task's number, the task is the ticket itself: its `change_task` values close it, whatever the switch says.

What the change and the task get, one note each step:

```
[AAP] Deploying this VM (AAP job 412 by jdoe):
vm_name: ab01web05   (from description)
template: rhel9-gold   (from description)
cluster: PROD-CL01   (from default (vm_deploy_defaults))
...
[AAP] VM ab01web05 deployed (AAP job 412 by jdoe).
IP address: 192.0.2.25
Customization: succeeded
vCenter: vc01.example.mil, datacenter DC01
Cluster: PROD-CL01, host esx01.example.mil
...
```

The job's output says it too: *"Notes on CHG0001234 and CTASK0001235. CHG0001234 is never closed (no close values for change_request); its tasks found are left open (vm_deploy_close_tasks: false)."*

**What you can leave alone for now:** sites and name rules (one place to build), plain words and GenAI (requests as `key: value` lines), network boot, the server's service, the scheduled pickup, the approval workflow. Each has its own section; the last chapter has every setting with its default, an example, and whether it is part of this setup.

# 2. Set it up in AAP, step by step

Do the steps in order: each one needs the ones before it. Steps 1 to 7 give you a deploy job that works by hand; steps 8 to 11 are optional and can wait until it does.

| Step | What | Needed |
|---|---|---|
| 1 | The execution environment has the VMware collection | always |
| 2 | A vCenter account with the role the deploy needs | always |
| 3 | The templates in vCenter are ready to be cloned | always |
| 4 | The credentials in AAP | always |
| 5 | The deploy settings finder: the names of your clusters, templates, folders | recommended |
| 6 | The deploy job template, its survey and its Variables | always |
| 7 | The first runs: a dry run, then a real one | always |
| 8 | The approval workflow: propose, approve, build | recommended |
| 9 | The scheduled pickup | optional |
| 10 | The server's service: its notes, its AAP inventory | optional |
| 11 | Launch it from another tool (AAP's API) | optional |

## 1. The execution environment

The deploy runs on the AAP side and talks to vCenter and ServiceNow; it never logs in to the new VM. Its execution environment (EE) needs Red Hat's certified collection `vmware.vmware` (2.11 or later) and its Python libraries (pyVmomi and the vSphere Automation SDK) - the same EE as the other VMware jobs. On the AAP server, as the user AAP runs as:

```shell
podman images | grep -i ee-
podman run --rm <the EE image your templates use> ansible-galaxy collection list vmware.vmware
podman run --rm <the EE image your templates use> python3 -c "import pyVmomi, vmware.vapi;
  print('ok')"
```

- **Both answer:** nothing to do.
- **Not there:** build an EE that has it, with `ansible-builder`, from your AAP version's `ee-minimal` image (on an air-gapped network, pull the collection from your Private Automation Hub). Then in AAP: **Execution Environments > Add** (the image you pushed). The all-in-one EE has it, with everything else the site's jobs need: EXECUTION_ENVIRONMENT.md (also a PDF).

execution-environment.yml

```
version: 3
images:
  base_image:
    name: registry.redhat.io/ansible-automation-platform-27/ee-minimal-rhel9:latest
dependencies:
  galaxy:
    collections:
      - name: vmware.vmware
```

Never add a `collections/requirements.yml` to this repository for it: AAP would try to download from Galaxy at every project sync, and on an air-gapped network every sync would fail. vCenter 7 or 8 is needed (the jobs also use vCenter's REST API).

## 2. The vCenter account and its role

A service account (e.g. `svc-aap-deploy@vsphere.local`) with a vCenter role that has only what the deploy needs. Give it the role on the **templates**, the **target clusters**, their **datastores**, the target **VM folders** and the **staging port group** - with **Propagate to children** ticked:

| Privilege group | Privileges |
|---|---|
| Virtual machine > Provisioning | Deploy template, Customize guest, Read customization specifications |
| Virtual machine > Inventory | Create from existing |
| Virtual machine > Change configuration | Change CPU count, Change memory, Set annotation, Add new disk, Extend virtual disk, Advanced configuration |
| Virtual machine > Interaction | Power on |
| Resource | Assign virtual machine to resource pool |
| Datastore | Allocate space |
| Network | Assign network |

- **A template with a vTPM or encrypted disks** (Windows Server 2022 / Windows 11 templates often have a vTPM, and a vTPM makes the VM encrypted) also needs *Cryptographic operations* > *Clone* (on the template), *Encrypt new* and *Register VM* (on the target VM folder), *Add disk* (on the VMs - for a ticket that adds a disk) and, only while the hosts' encryption mode is not on yet, *Register host* (on the clusters). Not *Manage keys*, *Manage KMS*, *Decrypt* or *Direct Access*. The hosts must reach the key provider (Native Key Provider or KMS) the template's keys come from.
- **Several vCenters** (`vmware_vcenters`, e.g. in Enhanced Linked Mode): the deploy logs in to **every** one of them with the same account - it checks the new name is free in all of them. Give the role as a **Global Permission** (vSphere Client: **Administration > Access Control > Global Permissions**, *Propagate to children*); read-only is enough on a vCenter it only reads. A `vsphere.local` account works only on the vCenters of its own single sign-on domain: `name@vsphere.local`, with the domain.
- **The hosts:** the deploy reads the ESXi hosts of the cluster it builds in, and the host the template is on. A role on a cluster without *Propagate to children* gives `lacks the privilege System.View on host-...`.

A privilege that is missing is named in the job's error (`lacks the privilege ...`), with the object.

## 3. The templates in vCenter

Each template a ticket may ask for:

- **VMware Tools** installed and running (open-vm-tools on Linux): the customization and the IP address come through it.
- **Linux with cloud-init:** VMware guest customization allowed (`disable_vmware_customization: false`).
- **Windows:** a **customization specification** in vCenter (*Policies and Profiles* > *VM Customization Specifications*) that runs Sysprep; you name it per template in `vm_deploy_template_specs`. The job sets the computer name to the VM's name.
- **Its network adapter** on your staging VLAN, with **Connect at power on** ticked. The new VM keeps the template's network; DHCP on that VLAN gives it its address.
- **In every vCenter** a site may build in, under the **same name**.
- **An empty template** (no operating system: it boots from the network into MECM, or from a CD into a kickstart) is set up differently: *Empty templates*, in the chapter *Where the VM goes*.

## 4. The credentials

AAP: **Automation Execution > Infrastructure > Credentials > Create credential**.

| Credential (type) | What it holds | Attach it to |
|---|---|---|
| **VMware vCenter** (built in) | vCenter Host (e.g. `vc01.example.mil`), the account of step 2. The job gets `VMWARE_HOST`, `VMWARE_USER`, `VMWARE_PASSWORD` | the deploy template, the settings finder |
| **ServiceNow API** (`aap/credential_types/servicenow_api.yml`, the one the ticket jobs use) | the instance URL and an API account with the `itil` role: it reads the ticket's table (incident, change task, change, catalog task) and the change, writes work notes and comments, and - when you close tasks - their state and close fields | the deploy template, the pickup |
| **ACT model key** (`aap/credential_types/act_model_key.yml`) | the GenAI key | the deploy template - only with GenAI on (`vm_deploy_genai`) |
| **Red Hat Ansible Automation Platform** (built in) | AAP's own address and an AAP user | the deploy template - only for a service with an AAP inventory (step 10); the pickup - its own user (step 9) |
| **SMTP relay** (`aap/credential_types/smtp_relay.yml`) | the mail relay's login | only when the relay needs one for `report_email_to` |

- **vCenter's certificate:** the jobs check it. If the EE does not trust your CA, a job fails with `certificate verify failed`: add your CA to the EE. Until then `vmware_validate_certs: false` in the template's Variables turns the check off (not recommended).

- **ServiceNow first:** set it up and test it with the *ServiceNow - test ticket* template (SERVICENOW_SETUP.md) before the deploy: the same credential, the same account.
- Nothing about vCenter, ServiceNow or AAP's passwords goes in Git.

## 5. Find the names: the deploy settings finder

Not sure what to write for `cluster:`, `datacenter:`, `vcenter:`, the template or a folder? Run the read-only **VM - deploy settings finder** first. For every vCenter (the credential's and `vmware_vcenters`) and datacenter it lists:

- the **clusters and standalone hosts**: hosts usable, memory used, the shared datastores with their free space (the ones automatic placement can use), the networks;
- the **templates**: where each one is (datacenter, cluster, folder), guest OS, CPU, memory, disks, network;
- the **VM folders**, as paths below the datacenter (what `folder:` takes), and the customization specifications;
- what each deploy setting means, with an example from your vCenter;
- what to look at first: a template in several datacenters (each name rule then needs its `datacenter:`), a datacenter with no template, a cluster with no shared datastore;
- **the lines to paste** into the deploy template's Variables: `vm_deploy_templates`, the template catalog for GenAI, and one `vm_deploy_name_rules` line per cluster with its datacenter (and vCenter, with several). Change `CHANGE-ME` to the part of the VM names that goes to that cluster, and delete the lines you do not need.

**Set it up: Templates > Create template > Create job template**: Name `VM - deploy settings finder`; Inventory: any; Project: site-automation; Playbook `playbooks/vm_deploy_inventory.yml`; Execution environment: your VMware one; Credentials: **VMware vCenter** (read-only is enough). Optional Variables: `report_email_to` (the email has the same tables), `vmware_datacenter` (one datacenter only).

**Launch it.** The paste lines are at the end of the output (and in the email, and in the job's artifacts as `vm_deploy_inventory.paste`). It changes nothing.

## 6. The deploy job template

**Templates > Create template > Create job template:**

- **Name:** `VM - deploy from ServiceNow ticket`
- **Inventory:** any (the job runs on the AAP side). **Project:** site-automation. **Playbook:** `playbooks/vm_deploy_from_ticket.yml`
- **Execution environment:** your VMware one (step 1)
- **Credentials: VMware vCenter** and **ServiceNow API** (step 4) - and, only when you use them, the ACT model key (GenAI) and the Red Hat Ansible Automation Platform one (a service's AAP inventory)
- **Limit:** empty, and not prompted (a Limit would make the job skip without doing anything)
- **Prompt on launch** ticked next to **Job type** (for the dry run) and next to **Variables** (the approval workflow hands each step its `vm_deploy_phase`)
- **Timeout:** room for every VM of a ticket: a template with guest customization may wait up to 25 minutes per VM (`vm_deploy_wait_customization` 900 s + `vm_deploy_wait_ip` 600 s); `0` = none
- **Variables:** the standard setup (chapter 1) with your names - or a configuration from *Ready-made configurations*. `aap/extra_vars/vm_deploy_from_ticket.yml` has every setting, commented, ready to

paste.

**The survey** (**Survey > Create question**, then switch the survey **on**):

| Question | Variable | Answer type | Required | What it is |
|---|---|---|---|---|
| *Ticket number* | `vm_deploy_ticket` | Text | **yes** | the change (`CHG...`), its task (`CTASK...`), an incident or a catalog task |
| *Service* | `vm_deploy_service` | Multiple choice: the names in `vm_deploy_services` | **no** | optional: what the server is for - words in the VM's notes and the AAP inventory each VM joins (step 10) |

**The *Service* question is optional.** Left empty - or not on the survey at all - a run is exactly what it was before the service existed: no words added to the VM's notes, no AAP inventory, no AAP credential needed. Add it when you set up step 10.

Launched through the workflow (step 8), the **workflow's** survey asks these questions and hands the answers to each step: put them there.

**What you should see:** the template saved, with two credentials (or more), and a survey that is on.

## 7. The first runs: a dry run, then a real one

1. **A test change, not yet approved:** in your test instance, a change whose description asks for a VM (`vm_name: ab01web05`, `template: rhel9-gold`, `cpu: 2`, `memory_gb: 8` - or plain words, with `vm_deploy_plain_words: true`), approval **Requested**, state **Assess**; a change task under it whose short description has your task's words (`vm_deploy_notes_tasks_query`).
2. **Launch the deploy template with the change's number as Check.** You should see: *"is not ready to build: approval is 'Requested', not Approved; state is 'Assess', not Implement"*. Those are the labels your instance uses; others than *Approved* / *Implement*: `vm_deploy_ready_by_table`.
3. **Approve the change, move it to Implement, Check again.** You should see each VM, each value with *"(from ...)"* - where it came from - where each would go (vCenter, cluster, datastore, folder), and *"Notes on CHG... and CTASK.... CHG... is never closed ..."*: who gets the notes, what is closed. Nothing is written on the ticket.
4. **Run it for real** (Job type **Run**): the VMs are built; the notes are on the change and the task; the task is closed or left open - as you set it. The job's report (and its email) says the same.
5. **Run it again:** the VMs already built are reported, not built twice. Cancel a run of several VMs while one is building, then run again: that one is finished in place, the rest are built.

Run a dry run first for every new kind of request - a new template, a new site, a new way of writing it.

## 8. The approval workflow: propose, approve, build

One launch: the deploy job checks and places (*propose*) and writes the plan on the ticket, a person approves it in AAP, then the deploy job builds (*build*) - *Report first, then build* in the next chapter says what each step does.

1. **The deploy template** (step 6) has **Prompt on launch** ticked next to **Variables**.
2. **Templates > Create template > Create workflow job template:** name `VM - deploy from ServiceNow ticket (workflow)`; a **survey** with the question *Ticket number*, variable `vm_deploy_ticket` (and, when

you use it, *Service*, `vm_deploy_service`) - the workflow passes the answers to every step. Save, then open the **Workflow Visualizer**:

- **Start › node 1:** job template `VM - deploy from ServiceNow ticket`; its prompt, Variables: `vm_deploy_phase: propose`
- **node 1 › (On success) › node 2: Approval**, name `Build the VM on the ticket?`, a timeout of, e.g., 8 hours
- **node 2 › (On success = approved) › node 3:** job template `VM - deploy from ServiceNow ticket`; Variables: `vm_deploy_phase: build`
- Nothing on *On failure*: a question for the requester, a refusal or a denied approval ends it.

3. **The approvers:** the workflow > **User access** > **Add roles** > the **Approve** role. Put their address in `report_email_to` on the deploy template: the propose step's email is what they read.
4. **Launch the workflow** with a test ticket. You should see *"[AAP] Checked - ready to build … it would go to …"* on the ticket and in the email; approve in **Automation Execution > Workflow Approvals**; the VMs are built.

- **The change's approval can stand in** for AAP's: for changes read by `key: value` lines or plain words, a workflow with the deploy job alone (no approval node) is enough - the change was approved for its words. With GenAI on, keep AAP's approval: the change was not approved for GenAI's reading.
- **Who can change the request:** anyone who can comment on the ticket can change it, and an incident or a task has no approval of its own. The person who launches the job is the control, and the "Deploying" note records exactly what was built. When a schedule picks tickets up (step 9), the workflow's approval is that control.
- **The plan the person approved is what is built:** the propose step's note carries a hash of the ticket's text (and its change's); a ticket edited while the approval waited is not built (*"was changed after the approval"*) - run the workflow from the start.

## 9. The scheduled pickup

`playbooks/vm_deploy_poll.yml` runs on a schedule - every 15 minutes, for example - and starts the deploy workflow for each new VM request in ServiceNow. Nobody types ticket numbers; nobody's approval is skipped. It builds nothing itself.

**Each run:**

1. It asks ServiceNow for tickets that are **active**, in the states `vm_deploy_poll_states`, **assigned to** `vm_deploy_assignment_group`, and **marked as VM requests**: the short description starts with `vm_deploy_poll_prefix` ("VM request", in any case), and/or they match `vm_deploy_poll_query`. Oldest first.
2. It skips every ticket that already has a note of AAP's on it (`[AAP] ...`): AAP has it already.
3. For each of the others, at most `vm_deploy_poll_max` (3) per run: it writes `[AAP] Picked up by AAP job ... - starting the workflow ...` on the ticket and reads it back (if the account could not see its own note, the next run would pick the ticket up again - so the job stops there, red); it starts the workflow `vm_deploy_poll_workflow` with the ticket number; it writes the workflow's job number and link on the ticket.
4. The workflow then does what it does when you start it by hand - and builds only after the approval.

It is green when it ran - also when there was nothing new - and red when ServiceNow or AAP did not answer or a workflow could not be started. It emails (`report_email_to`) only when it started something or

something failed.

| Guardrail | What it does |
|---|---|
| The approval stays | the job only starts the workflow; a person approves every VM before it is built |
| Only marked tickets | the group is required, and a VM request must be marked (the prefix or the query): an ordinary ticket of the group is never picked up. A query with `^NQ` (a second query that would bypass the group) is refused |
| A limit per run | `vm_deploy_poll_max` (3): a flood of tickets cannot start 50 workflows; the rest wait for the next run |
| Once per ticket | the "Picked up" note, read back before the workflow starts; a ticket with any note of AAP's is never picked up again |
| An off switch | `vm_deploy_poll_enabled: false`: the schedule runs, picks up nothing and says so |
| A dry run | `vm_deploy_poll_dry_run: true` (or Job type Check): lists what it would pick up - no note, no workflow, no email |
| Its own AAP user | the job starts the workflow as an AAP user that may do nothing else |

**Set it up:**

1. **The workflow works by hand first** (step 8), tested with a ticket.
2. **An AAP user that may only start the workflow: Access Management > Users > Create user**, e.g. `svc_aap_pickup` (a normal user, not an administrator). Then the workflow > **User access** > **Add roles** > that user > the **Execute** role. Nothing else.
3. **Its credential:** type **Red Hat Ansible Automation Platform**: AAP's own address (e.g. `https://aap.example.mil`, the one in your browser), the user name and password from step 2 (or a token of that user), **Verify SSL** ticked. The job gets them as `CONTROLLER_HOST`, `CONTROLLER_USERNAME`, `CONTROLLER_PASSWORD` (or `CONTROLLER_OAUTH_TOKEN`); the `AAP_*` names newer AAP versions add work too. If the EE does not trust AAP's certificate, set `vm_deploy_poll_aap_ca_path` to its CA file rather than unticking Verify SSL.
4. **Templates > Create template > Create job template:** Name `VM - pick up deploy tickets`; Inventory: any; Project: site-automation; Playbook `playbooks/vm_deploy_poll.yml`; Execution environment: any (it only talks to ServiceNow and AAP); Credentials: **ServiceNow API** and the **Red Hat Ansible Automation Platform** one from step 3; **Prompt on launch** next to **Job type**; **Concurrent jobs** unticked (two runs at once could pick up the same ticket); Variables, e.g. for incidents:

YAML

```yaml
vm_deploy_assignment_group: Server Team     # REQUIRED - the same group as the deploy job
vm_deploy_poll_prefix: "VM request"         # tickets whose short description starts with it
vm_deploy_poll_max: 3
vm_deploy_poll_dry_run: true                # the first week: list only
report_email_to: vmteam@example.mil
```

5. **Launch it once as Check:** it lists the tickets it would pick up and says whether AAP can start the workflow (`AAP: workflow "..." (id ...) can be started`).
6. **The schedule:** the template > **Schedules** > **Create schedule**: a name (`every 15 minutes`), a start time, **Repeat frequency** Minute, every **15**. Save.

7. **A week of dry runs:** read the jobs' output - are these the tickets you expect? Then delete `vm_deploy_poll_dry_run: true` from the Variables. From the next run, tickets are picked up.

Tell the requesters to start the short description with **VM request** (or ask the ServiceNow team for a category or a catalog item, and use `vm_deploy_poll_query`).

**Change tasks whose change is approved and in Implement** - the pickup's Variables:

```yaml
vm_deploy_poll_tables: [change_task]
vm_deploy_poll_prefix: "VM request"     # how your VM change tasks' short description starts
vm_deploy_poll_query: "change_request.approval=approved^change_request.state=-1"
vm_deploy_poll_states: ["1"]            # the task's own state VALUES to pick up (1 = Open)
```

- The query reaches into the task's change (ServiceNow's *dot-walking*): only tasks whose change is approved (`approved`) and in Implement (`-1`) are picked up. These are **values**, not labels: build the filter in the change task list (*Change request.Approval* is *Approved* ...), then right-click the filter and *Copy query*.
- Tasks that do not start with one phrase, e.g. *Task to deploy a staging VM*: no prefix, and the words in the query - `LIKE` is *contains* (any case), `^OR` adds another wording: `vm_deploy_poll_prefix: ""` and `vm_deploy_poll_query: "short_descriptionLIKEto deploy^ORshort_descriptionLIKEdeploy VM^change_request.approval=approved^change_request.state=-1"`. Search for a phrase, not a short word alone: `LIKE` also finds it inside other words (`app` is in *h**app**y*).
- `vm_deploy_poll_states` are the **task's** state values: a state your instance added (e.g. one shown as *Implementation*) has its own value - copy it the same way.
- A task that has a note of AAP's on it is never picked up, so a change the deploy built when launched with the change itself is not built again from its task.

The pickup's messages: *When something goes wrong*, *The scheduled pickup*.

## 10. The server's service: its notes, its AAP inventory

**Optional.** Whoever launches the deploy may pick what the new server is for - `sql`, `dc`, `web` ... - in the second survey question, *Service*. Left empty, nothing of this step happens. Picked, it does two things for every VM of the ticket:

- **The VM's notes in vCenter** get the service's words on a line of their own, under the ticket's line, the template and the job (*Where the notes go*, below): *Service: SQL Server*.
- **With an inventory**, each VM joins that AAP inventory and group right after its own build, before its note on the ticket - a job stopped between two VMs leaves each VM it built in AAP. It is added by its name, with `deployed_ticket`, `deployed_service`, `deployed_on` and (when it has one) `deployed_ip` as its variables. Never `ansible_host`: AAP connects by the name, as for your other servers. The result note on the ticket says it: *AAP inventory Windows servers, group win_sql: added as sql01.example.mil*.

```yaml
vm_deploy_services:
  sql:  {inventory: Windows servers, group: win_sql, note: SQL Server}
  dc:   {inventory: Windows servers, group: win_dc, note: Domain controller}
  web:  {inventory: Linux servers, group: web}                  # note left out = "Service: web."
  test: {note: Test server - delete after 30 days}              # the notes only, no inventory
vm_deploy_inventory_name: "{name}.example.mil"                  # the way your inventory writes
  server names
```

**Set it up:**

1. **The inventories and groups must already exist in AAP** - the job never creates one. A name that is not there (or a smart or constructed inventory, which takes no hosts) stops the job **before anything is built or written on the ticket**, and says what to create.
2. **An AAP user with the Admin role** on each of those inventories (the inventory > **User access** > **Add roles**), and a credential of type **Red Hat Ansible Automation Platform** for it: AAP's address, that user (or its token), **Verify SSL** ticked (a CA file the EE does not trust: `vm_deploy_aap_ca_path`). Attach it to the **deploy** job template. Not the pickup's user: it may only Execute the workflow.
3. `vm_deploy_services` in the deploy template's Variables (or `all.yml`): your services, as above. Only services with an `inventory` need steps 1 and 2; a service with only a `note` needs neither.
4. **The survey question** - on the workflow when you launch the workflow, else on the job template: *Service*, variable `vm_deploy_service`, **Multiple choice**, its choices exactly the names in `vm_deploy_services` (`sql`, `dc`, `web`, `test`), **not required**. A name that is not in the list is refused before anything is done. The scheduled pickup starts the workflow without it: those VMs get no service.
5. `vm_deploy_inventory_name`**:** write it the way your inventory already names servers (`{name}` = the VM's name, lower case unless `vm_deploy_inventory_lowercase: false`) - or the same server shows up twice. A service's own `name:` wins (e.g. Windows servers in another domain: `sql: {inventory: Windows servers, group: win_sql, name: "{name}.win.example.mil"}`).
6. **A dry run** (Check) of a test ticket with a service. You should see: *AAP inventory Linux servers, group web: would be added as app01.example.mil* - nothing added.

**Watch for an inventory filled by a source** - e.g. the inventory file from your Ansible server, as a project source - with **Overwrite** ticked. At its next sync AAP may take a host the deploy added out of the groups the source fills. The job warns when it sees one: untick **Overwrite** on that source, or add the new server to the file as well.

**AAP does not take a VM** (the user lacks the Admin role, AAP is down): a **WARNING** on the ticket and in the report - *NOT added ... Add sql01.example.mil by hand* - never a failure: the VM is built and the ticket closed as usual. Run the job again: the VM is left as it is, and the host is added. A run again never adds a host twice.

## 11. Launch it from another tool (AAP's API)

A web front end (or any tool that can send HTTPS) starts the **workflow** with the change's (or the task's) number - the only answer of its survey; the request stays on the ticket, the tool needs no ServiceNow or vCenter access:

```
POST https://<aap>/api/controller/v2/workflow_job_templates/<id>/launch/        (AAP 2.4:
  /api/v2/...)
Authorization: Bearer <a token of an AAP user that may only Execute this workflow>
Content-Type: application/json

{"extra_vars": {"vm_deploy_ticket": "CHG0031234"}}
```

AAP answers `201` with the workflow job's `id`. With the survey on, AAP drops every variable that is not one of its questions: the tool cannot change the rules (add `"vm_deploy_service": "sql"` when the survey has *Service*). What the tool reads back:

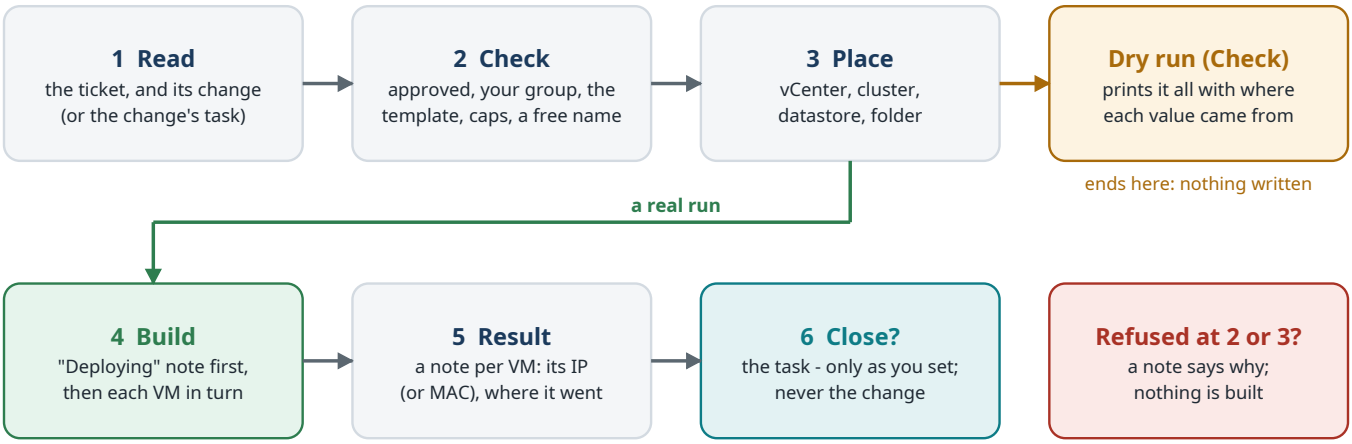
| What | Where |
|---|---|
| running, waiting, successful, failed | `GET .../workflow_jobs/<id>/` (`status`) |

| What | Where |
|---|---|
| each step and its job (propose, the approval, build); the approval waits as `workflow_approval`, `pending` | `GET .../workflow_jobs/<id>/workflow_nodes/` |
| the result, as data: `artifacts.vm_deploy` - `ticket`, `built`, `waiting_for_approval`, `ok`, `error`, `spec` and `sources`, `vms` (each: name, `ok`, `error`, `aap_inventory`, and `vm`: vCenter, datacenter, cluster, IP, MAC, disks, power), `service` | `GET .../jobs/<job id>/` |
| the report as text, as the job prints it: `artifacts.vm_report.lines` | the same |
| the whole output | `GET .../jobs/<job id>/stdout/?format=txt` |
| approve or deny the plan (an AAP user allowed to approve) | `POST .../workflow_approvals/<id>/approve/` (`/deny/`) |

The ticket's work notes stay the record (with `vm_deploy_notes_also: [parent, tasks]`, on the change and its task). A request the rules refuse fails its job with the reason in the output (*Not deployed: ...*) and on the ticket. A workflow cannot run as Check: for a preview, launch the job template `VM - deploy from ServiceNow ticket` with `"job_type": "check"` (Prompt on launch for Job type).

# 3. How a run works

## The flow, step by step




*One run of the deploy job. Check (a dry run) stops after the placement; a real run writes "Deploying" before it builds anything, and closes only what you set.*

0. **The service** (only when the launch names one): it must be one of `vm_deploy_services`, and its AAP inventory and group must be there and open to the AAP credential's user - checked **before the ticket is read**: refused, nothing is read, written or built.
1. **Read.** The job reads the ticket you launched it with - and, for a change task, its change (for a change: its open tasks of your group, for the notes). It reads `key: value` lines, plain words (`vm_deploy_plain_words`) and, when it is on, has GenAI copy the request out (`vm_deploy_genai`).
2. **Check.** The change is approved and in Implement (`vm_deploy_ready_by_table`); the ticket is your group's; the template is allowed; CPU, memory and disks are within the caps; each name is valid and free in every vCenter. Several VMs: each is also checked in vCenter (its name, template, place, customization) before the first is built. Anything wrong: a note on the ticket says what - **nothing is built**.
3. **Place.** For each VM: its vCenter, datacenter, cluster, datastore and folder (chapter *Where the VM goes*). **A dry run (Check) ends here**: it prints the request, where each value came from and where each VM would go, and writes nothing on the ticket.
4. **Build.** "[AAP] Deploying ..." on the ticket first (no note = nothing built), then each VM in turn: clone, CPU, memory, disks, customization (or network boot), power on, wait for its IP address - then, with a service that has an AAP inventory, the VM joins it. Each VM carries the ticket on the first line of its notes from the clone on (*CTASK0001235 - building 2026-10-20*, then *built*): a job that stopped is picked up where it stopped.
5. **Result.** One note per VM, written as soon as that VM is done: "[AAP] VM ab01web05 deployed" with its IP (or MAC for a network boot) - and, with a service, the AAP inventory it joined.
6. **Close - only what you said.** The task closes only when you set its close values and every VM is built (*Closing the task*, below). The change is never closed.

**Each VM, in detail:**

- It is cloned from the template, **powered off**, with its notes (*Where the notes go*, below): the ticket's line first, *CTASK0010009 - building 2026-10-18*.

- CPU, memory and disks are set: `disks_gb: 120, 200` grows disk 1 to 120 GB (never shrinks it) and adds a 200 GB disk (thin).
- **Guest customization:** Linux - the host name is the VM's name, every network adapter on DHCP, the domain `vm_deploy_domain`; Windows - the template's customization specification (`vm_deploy_template_specs`), its computer name set to the VM's name. An empty template gets none (*Empty templates*).
- It is powered on; the job waits for vCenter's customization result (`vm_deploy_wait_customization`, 900 s) and for an IPv4 address from VMware Tools (`vm_deploy_wait_ip`, 600 s).
- Its notes' first line becomes *CTASK0010009 - built 2026-10-18* (*failed* when its customization failed), and its note goes on the ticket: IP address, customization result, vCenter, cluster, host, datastore, folder, size. On a failure: what failed, and whether a (partly set up) VM exists.
- The network is the template's: its adapter is on your staging VLAN, and so is the new VM.

## Report first, then build

Three ways to see what would be built before anything is:

1. **Check (a dry run)** - the simplest. Launch the deploy template with *Job type* **Check**. It reads the change, checks everything, chooses where each VM would go, and prints it all with where each value came from. It **writes nothing** on the ticket. Then launch it again as **Run** to build.
2. **The approval workflow** (`VM - deploy from ServiceNow ticket (workflow)`, set up in step 8) - one launch:
   - step 1 (`vm_deploy_phase: propose`) checks and places, and writes *"[AAP] Checked - ready to build after the approval ... It would go to ..."* on the change and its task, and emails it;
   - a person clicks **Approve** in AAP (*Deny* = nothing built);
   - step 3 (`vm_deploy_phase: build`) reads the ticket again, checks again, and builds;
   - step 3 builds **only what was approved**: the propose step's note carries a hash of the ticket's text (and its change's). A ticket edited while the approval waited - a new comment, an edited description, its change - is not built: *"was changed after the approval"*. Run the workflow from the start: it is checked and approved again.
3. **GenAI without the workflow** - with `vm_deploy_genai: true`, the first run posts GenAI's proposal and stops; the next run builds that proposal.

The workflow, from the request to the VM:

```
 1. Requester ── writes the ticket in ServiceNow (plain words, or key: value lines)
 2. Operator  ── launches the workflow in AAP with the ticket number          (ONE launch)

── workflow step 1: PROPOSE (the deploy job, vm_deploy_phase: propose) ──────────────────
 3. AAP → ServiceNow: reads the ticket (and its change)
 4. AAP → GenAI:      "copy out what this ticket asks for"       (only when GenAI is on and the
 5. GenAI → AAP:      names, OS as written, CPU, memory, disks    lines and words leave a gap)
 6. AAP checks every value really is in the ticket, and finds the template from the OS words
       something AAP needs is missing?  → AAP → ServiceNow: its questions to the requester (a
         comment);
                                          the workflow STOPS
 7. AAP checks the rules (templates, site, caps, a free name, the group) and finds the cluster and
    datastore (read-only)
 8. AAP → ServiceNow: "[AAP] Checked - ready to build: ... it would go to ..." (+ the GenAI
   proposal)
    AAP → email: the same, to the approver.                       NOTHING IS BUILT

── workflow step 2: APPROVAL ──────────────────────────────────────────────────────────────
 9. Approver  ── reads the email (or the ticket), clicks Approve in AAP     (Deny = nothing built)

── workflow step 3: BUILD (the deploy job, vm_deploy_phase: build) ──────────────────────
10. AAP → ServiceNow: reads the ticket again   (GenAI is NOT asked again; a ticket changed since
                      the approval = nothing built)
11. AAP checks the rules again; AAP → ServiceNow: "[AAP] Deploying ..." (refused = nothing built)
12. AAP → vCenter:    each VM: clone, CPU / memory / disks, customize (or network boot), power on
    AAP → AAP:        the VM joins its service's inventory (only with a service that has one)
13. AAP → ServiceNow: "[AAP] VM ... deployed" - IP (or MAC for MECM), where it went
14. AAP → email: the result
```

For a demo with no person in between: `vm_deploy_genai_approval: false` on the job template and launch the job itself - one run reads, checks and builds, when every value is traced to the ticket and within the rules; questions or a refusal still stop it.

## Which number to launch with

The deploy template's survey asks for one ticket number. **Launch with the change** (`CHG...`): the team that raises the change knows its number. A change task's number (`CTASK...`) works too, and is what the scheduled pickup uses. An incident (`INC...`), a catalog task (`SCTASK...`) or a request item (`RITM...`) works the same way, without a change. Your instance's own prefixes (`INCN...`, `CHGN...`) work as well: the job asks ServiceNow's task table which table a ticket is in.




*The same settings work for both. Only the ticket launched - and, launched with the change, its task with vm_deploy_close_tasks - is ever closed.*

| | Launched with the change (CHG...) | Launched with the change task (CTASK...) |
|---|---|---|
| Read | the change; its open tasks of `vm_deploy_assignment_group` are found | the task, then its change (the request may be in either) |
| Must be approved and in Implement | the change | the task's change |
| Group check | the change: `vm_deploy_assignment_group` (`""` = any group), or any group with `vm_deploy_assignment_group_by_table: {change_request: ""}` | the task: `vm_deploy_assignment_group` |
| Notes, with `vm_deploy_notes_also: [parent, tasks]` | the change **and** its task(s) found | the task **and** its change |
| Closed | the task(s), only with `vm_deploy_close_tasks: true` and a `change_task` close entry | the task, when there is a `change_task` close entry |
| Never closed | the change | the change |

- **How the change's task is found:** its open tasks assigned to `vm_deploy_assignment_group`, and/or matching `vm_deploy_notes_tasks_query` (its words) - at least one of the two. The standard setup uses the words alone (any group); with a group set, `vm_deploy_assignment_group_by_table: {change_request: ""}` lets the change itself be any group's while its task must be yours.
- **No task matches** (not created yet, closed, other words): the job still builds, notes the change only, and says so: *"No open task of CHG... matches short_descriptionLIKEto deploy: the notes are on the change only."*
- **Several tasks match** under one change: each gets the notes (and is closed, when closing is on).
- **Which text wins** when both the change and the task describe the VMs: the task's - from the least to the most, the change, the task's short description and description, its work notes, its comments. `vm_deploy_parent_wins: true` puts the change over the task's description (a work note or comment on the task still wins). A `key: value` line and plain words rank by where they are written, not how.
- An incident or a catalog task works the same way as a task, without a change: its own table's close entry (`incident`, `sc_task`) closes it.

## The change: what is read, what must be true

A common setup: the **change request** holds the requirements and is approved; the team gets a **change task** that may or may not repeat them. Whichever number the job is launched with, it reads both.

**The request from the change** - nothing to set:

- From a task, the job follows its link to its change - the field `change_request` (a change task's), else `parent` (`vm_deploy_parent_fields`) - and reads the change's description (`vm_deploy_parent_text_fields`; add `implementation_plan` if your changes keep the request there).
- Both the change's `key: value` lines and, with `vm_deploy_plain_words`, its **plain words** are read. What the task says itself wins: a task that only says *"deploy the VM of the change"* takes everything from the change; a task that says `memory_gb: 16` changes only the memory. The dry run says where each value came from: `vm_name: web01, web02 (from the words 'named web01 and web02' in the parent CHG0030001 description)`.

- Only a parent that is a **change** is read (`vm_deploy_parent_tables: [change_request]`; `[]` = never read a parent): an incident's parent incident, or a catalog task's request item, is not.
- The change's comments and work notes are not read (CAB notes are not a request), unless you list them in `vm_deploy_parent_text_fields`. GenAI (`vm_deploy_genai`) reads the change's text too.

**The change must be approved and in Implement** (`vm_deploy_ready_by_table`, on by default):

```yaml
vm_deploy_ready_by_table:
  change_request: {approval: Approved, state: Implement}
```

- Written as the change's form shows them (upper / lower case does not matter); a list allows any of them: `state: [Implement, Scheduled]`. Your instance's labels may differ - the refusal says what the change has: *"the change CHG0030001 (this ticket's parent) is not ready to build: approval is 'Requested', not Approved; state is 'Assess', not Implement"*.
- It applies to the change a task belongs to, and to a change that is itself the ticket. Other tables can have an entry too, e.g. `change_task: {state: [Open, Work in Progress]}`. `{}` = no check.
- It is checked by the dry run, by the workflow's propose step **and again right before the build**: a change sent back while the AAP approval waited is not built.
- A change that **cannot be read** (the ServiceNow account needs read access to `change_request`), or is **closed or canceled**, stops the build too.
- Not ready = nothing built, a note on the ticket says why. **Run the job again** once the change is ready (a ticket the schedule picked up is not picked up again).

**The change and its task, and which wins:**

```yaml
vm_deploy_parent_wins: true                 # only if the change must win over the task's
  description (default: the task wins)
vm_deploy_parent_text_fields: [description, comments]   # an answer written as a comment on the
  change counts too
vm_deploy_notes_also: [parent, tasks]       # every note on the change too - or, launched with the
  change, on its task
vm_deploy_notes_tasks_query: "short_descriptionLIKEto deploy"   # which of the change's tasks
  (with vm_deploy_assignment_group)
vm_deploy_assignment_group_by_table: {change_request: ""}   # launched with the change: it may be
  any group's
```

- **Which wins:** off (the default), the change is read first and the task's text wins over it. On, the task's short description and description are read first, then the change's - which wins where both say something; the task fills in only what the change does not say. Either way a work note on the task wins over both, and a comment over a work note.
- Launched with the **change**, the change itself must pass the rules for a ticket: approved and in Implement, and assigned to `vm_deploy_assignment_group` - or to **any group** with `vm_deploy_assignment_group_by_table: {change_request: ""}` (a change is often assigned to whoever owns it, not to your team). Its tasks that get the notes are still found by `vm_deploy_assignment_group` and/or `vm_deploy_notes_tasks_query`, and a ticket launched by itself of any other table (a change task, an incident) must still be your group's.
- **One change task for all the change's VMs** works as it is. With **one task per VM**, give each task its own `vm_name:` line: otherwise each task reads every name from the change, and the second task is refused

(the names are taken).

## Closing the task: pick one

| You want | Set this on the deploy template |
|---|---|
| **The task closed by hand** (the job never closes anything) | `vm_deploy_close_tasks: false` (the default). A `change_task` entry may stay: launched with the change, it is used only with the switch on |
| **The task closed when every VM is built** | `vm_deploy_close_tasks: true` and the `change_task` entry (below) |
| ... closed with another close code when a VM is built but something did not finish | add `vm_deploy_done_fields_issues_by_table` |
| ... but a network-boot (MECM) VM's task left open until someone checks the install | add `vm_deploy_done_fields_pxe: false` |
| **The change never closed** | keep `vm_deploy_done_fields: {}` (the default) and no `change_request` entry anywhere |

**Closed by hand** - every note still goes on the task, *"[AAP] VM ab01web05 deployed"* included; your team closes it:

The task closed by hand

```
vm_deploy_notes_also: [parent, tasks]
vm_deploy_close_tasks: false
```

**Closed when complete:**

The task closed when every VM is built

```
vm_deploy_notes_also: [parent, tasks]
vm_deploy_close_tasks: true
vm_deploy_done_fields_by_table:
  change_task: {state: "3", close_code: successful, close_notes: "VMs deployed by AAP"}
  sc_task: {state: "3", close_notes: "VM deployed by AAP"}        # a catalog task, launched by
    its number
vm_deploy_done_fields_issues_by_table:
  change_task: {close_code: successful_issues}
```

What the close settings mean:

- `vm_deploy_done_fields_by_table` - **the values** that close a ticket of a table (`change_task`, `sc_task`, `incident` ...), also with your own number prefixes. Its `change_task` entry closes a change task the job was launched with, on its own. These are your instance's **values**, not the labels: open a closed change task, right-click its header, *Show XML*, and copy what `<state>` and `<close_code>` hold.
- `vm_deploy_close_tasks` - **launched with the change**, close its task(s) too, with those same values. On without a `change_task` entry: nothing is closed, and the job warns *"vm_deploy_close_tasks is on, but vm_deploy_done_fields_by_table has no change_task entry"*.
- `vm_deploy_done_fields_issues_by_table` - laid **over** those values when every VM is built but something did not finish: no IPv4 address in time, no customization result, the host name not applied, a network adapter not connected, `guestinfo` not set. The task's notes say what. (A network-boot VM's *"no operating system yet"* is not an issue.)
- `vm_deploy_done_fields_pxe: false` - a VM from a network-boot template is powered on and still installing from MECM when the job ends: its task stays open for whoever checks the install.

- `vm_deploy_done_fields` (no table) - for **every** table without an entry, **a change too**: leave it `{}`.

Always:

- **Closed only when every VM is built.** One VM not built: the task stays open with a note per VM. Fix the cause and run the job again - the VMs already built are left as they are, one it started and did not finish is finished in place (never cloned again), the rest are built, then it closes. The work notes say which (*"Picking up where an earlier run for this ticket stopped: ..."*).
- **The close is sent after the result note, on its own**, and the job checks what ServiceNow kept. When ServiceNow refuses it (a mandatory field, a business rule): the VMs are built, the job stays green, and the task and the report say *"NOT closed - close it by hand: ..."* with ServiceNow's reason. Add the missing field to the close values. (A value that is not one of the field's choices is not caught: take the values from *Show XML*.)
- A **closed** task is never deployed again: close a task *Unsuccessful* by hand only when you give up.

## Where the notes go

| Note | Where it is written | Who reads it |
|---|---|---|
| `[AAP] Checked - ready to build ... it would go to ...` | work note (the propose step, or a GenAI proposal waiting for a person) | the approver |
| `[AAP] GenAI proposal ...` | work note: each value with the ticket words it came from | the approver |
| `[AAP] We cannot build this VM yet: ...` - the questions | **comment** (requesters see comments, not work notes) | the requester |
| `[AAP] Deploying ...` | work note, before anything is built | the record |
| `[AAP] VM ab01web05 deployed` / `NOT deployed` | work note, one per VM - with the service and its AAP inventory, when one was picked | the team |
| `[AAP] 1 of 2 VMs built; this ticket stays open` | work note | the team |
| `[AAP] Picked up by AAP job ...` | work note (the scheduled pickup) | the record |

- With `vm_deploy_notes_also: [parent, tasks]`, every one of them goes on the ticket the job was launched with **and** on the other one (the change, or its task). Only the launched ticket is closed - and the change's task with `vm_deploy_close_tasks`.
- The job's output says where its notes go and what it closes: *"Notes on CHG0001234 and CTASK0001235. CHG0001234 is never closed (no close values for change_request); its tasks found are closed too (vm_deploy_close_tasks)."* A dry run (Check) says the same, and writes nothing.
- A note the other ticket does not take (the account cannot update it) is a warning, not a failure.
- The requester answers a question with a **comment** on the ticket; a later comment wins over the description. Then run the job (or workflow) again.
- **In vCenter**, each VM's notes say which ticket built it, one line each:

```
CHG0001234 - built 2026-10-18
Template: rhel9-gold
AAP job 812 (operator1)
Service: SQL Server
Notes: the ticket's VM notes
```

The first line is the ticket's mark: a run of the ticket again finds its VMs by it (*Run it again*, below), and the *VM - VMs created* report reads the ticket and the template from it. *building* while the job builds it, *failed* when its customization failed; the date is the day it was marked (UTC). *Service* and *Notes* only when the ticket has them. Lines you add under them, or above, are kept by vCenter and change nothing; edit the first line and the job no longer knows the VM.

## Several VMs on one ticket

One ticket may ask for several VMs: one VM per name, the template, CPU, memory, disks and folder the same for each - unless each server has a line of its own that says its own (*A change laid out per server*, in *Plain words*).

```
I need 2 vms created named web-test01 and web-test02
both servers need:
Windows 2022
8 cpus
12gb of memory
storage 180GB disk and 50gb disk
Store in the App-Servers folder.
```

The names, as people write them: `vm_name: web01, web02` (a line), `Hostnames:` with one name per line below it (`- web01`, `1. web01` ...), and in plain words `named web01 and web02`, `named web01, web02 and web03`, `2 VMs: web01 and web02`.

- **Each VM is checked on its own**: its name (valid, not used in any vCenter), its name rule (so two names may go to two clusters), the caps (`vm_deploy_max_cpu` ... are per VM). One VM refused = none is built: the ticket says why. That holds in vCenter too: **every VM is checked there before the first is built** (read-only: its name is free, its template, cluster, datastore and folder are there, its customization is set up) - *"[AAP] Nothing was built: ... 1 of 3 cannot be: - web03: a VM named web03 already exists"*.
- **How many** - `2 VMs`, `two servers`, `both servers`, `3 virtual machines` - must match the names: *"2 VMs named web01"* is a question to the requester, nothing is built. A range (`web01 to web05`, `web01-05`) is never expanded: a question too - every name is written out.
- **At most** `vm_deploy_max_vms` **(10) per ticket.** More: refused, nothing built (split the ticket, or raise the setting).
- **Built one after the other.** Each VM is placed after the ones before it are there, so automatic placement sees them. The dry run and the propose step show where each would go as if alone: the build may place them differently.
- **On the ticket:** "[AAP] Deploying 2 VMs, one after the other", then one note per VM, written as soon as that VM is done ("[AAP] VM web01 deployed (...) - 1 of 2."). The ticket is **closed only when every VM is built**. With a service, every VM of the ticket gets the same one.
- **One fails, the others are still built.** The ticket stays open with a note per VM and one that sums it up: *"[AAP] 1 of 2 VMs built; this ticket stays open: - web01: built - web02: NOT deployed - ..."*. Fix the cause and **run the job again**: it picks up where it stopped (next).

- **The job takes as long as all its VMs.** A template with guest customization waits up to `vm_deploy_wait_customization` (900 s) and `vm_deploy_wait_ip` (600 s) **per VM**: ten such VMs can take hours. Give the deploy template a **Timeout** that allows it (0 = none), or keep tickets small. Network-boot (PXE / MECM) templates do not wait for either.

## Run it again: it picks up where it stopped

A job that stopped part way - a VM failed, the job was cancelled or timed out, AAP restarted - is run again for the same ticket, and **picks up where it stopped**. Each VM carries its ticket on the first line of its notes from the clone on (*CTASK0010009 - building 2026-10-18*, then *built* once it is up and its customization result is in), so the job knows its own VMs even when a result note never reached the ticket. VMs built by releases before this one carry *[site-automation ticket=CTASK0010009 state=built]* instead - read the same way, and their notes rewritten in the new layout when one is finished:

| The VM in vCenter | Run again |
|---|---|
| marked **built** for this ticket | left as it is: reported with its IP address (and added to its service's AAP inventory if it is not there yet) |
| marked **building** for this ticket (cloned, not finished) | **finished where it is**: the CPU, memory and disks it still lacks, the customization, the power on - never cloned again (already powered on: its result is read) |
| not there | built |
| there, but not this ticket's (another ticket's mark, or none) | refused, nothing built: *a VM named web02 already exists* |

The work notes say which: *"[AAP] Deploying 3 VMs ... Picking up where an earlier run for this ticket stopped: - web01: built already - left as it is - web02: started, not finished - finished in place - web03: to build"*, then each VM's own note (*"Picked up where an earlier run for this ticket stopped: finished in place."*). The ticket is closed once all of them are built.

- One VM tickets work the same: run again, the VM it built is reported, not refused (and the ticket closed, when close fields are set). To build a test VM again, delete it in vCenter first.
- VMs built by an earlier release (no mark; their notes begin *"Deployed from TEMPLATE for CTASK0010009 by"* - the ticket's number written anywhere else does not count): powered on or with the ticket's *"[AAP] VM web01 deployed"* note - left as they are; powered off - finished in place.
- Built, but its **guest customization failed** (marked *failed*): refused - check it, delete it, run again.
- `vm_deploy_resume: false`: the way of earlier releases - a VM the ticket's own *"[AAP] VM web01 deployed"* note names is left as it is; any other VM with the name is refused, and one started is never finished.
- The PXE / MECM case works the same: each VM gets its own MAC address on its own note.

# 4. The request: what the ticket says

## What the requester writes

In the ticket's description (or the change's), or a comment - `key: value` lines:

```
vm_name: ab01web05, ab01web06
template: rhel9-gold
cpu: 4
memory_gb: 16
disks_gb: 120, 200
site: SiteA
```

Or, with `vm_deploy_plain_words: true`, plain words:

```
Please build 2 Red Hat 9 servers named ab01web05 and ab01web06, 4 vCPU, 16 GB RAM,
storage 120GB and 200GB, Location: North Campus.
```

Or a change laid out one server per line - each line is its VM's own (size, disks, site):

```
a. spin up new 2025 VM / ab01web05 / CPU : CORE.4 / RAM : 16 GB. / HDD'S C: /120 Location: SiteA
a. spin up new 2025 VM / ab01web06 / CPU : CORE.8 / RAM : 32 GB. / HDD'S C: /120 Location: SiteB
```

- **Never a guess:** two answers for one value (`RHEL 9 or RHEL 8`), a count that does not match the names, a range (`web01 to web05`) - a question to the requester (a comment); nothing is built.
- **What wins:** the place it is written - a comment over a work note, both over the description, the ticket over its change. In the same place, a `key: value` line over the words. GenAI's proposal fills only what the ticket leaves open, then the defaults. Two answers in the place that wins are a question, never settled by GenAI or a default (GenAI may still lay out disks the words list: `120gb disk`, `50gb disk`).
- At most `vm_deploy_max_vms` (10) VMs per ticket; built one after the other.
- The requester never picks the service: whoever launches the job does (the *Service* survey question).

## The lines: key: value

`key: value` lines, in the ticket's **description** or in a **comment / work note**. A later comment overrides an earlier value (to correct a request, add a comment with the new line):

```
vm_name: app01
template: rhel9-gold
cpu: 4
memory_gb: 16
```

| Key | Meaning | Not given |
|---|---|---|
| `vm_name` | the VM's name in vCenter and its host name - or **several names** (`vm_name: web01, web02`): one VM each | **required** |
| `template` | the template, by its name in vCenter (or a short name, `vm_deploy_template_aliases`) | **required** (unless `vm_deploy_defaults` has one, or the words name the OS) |

| Key | Meaning | Not given |
|---|---|---|
| `cpu`, `memory_gb` | vCPUs, memory in GB | the template's |
| `disks_gb` | disk sizes in GB, system disk first: `disks_gb: 120, 200` grows disk 1 to 120 GB (never shrinks it) and adds one 200 GB disk (thin) | the template's disks |
| `site` | which site: its cluster, datastore, folder (and vCenter) come from `vm_deploy_sites` | the ticket's **Location** field, when it is one of the sites |
| `cluster`, `datastore`, `folder` | where it goes (`datastore` may be a datastore cluster) - overrides the site's. `folder` as a path below the VM folder (`Linux/App`) is that path exactly; a name alone (`App`) is the one folder of that name; the folder the VM goes in - the template's too, when none is given - must be one of `vm_deploy_folders` (when you set that list) | the site's, else `vm_deploy_defaults`, else the template's own |
| `domain` | the DNS domain set in the guest (Linux) | `vm_deploy_domain` |
| `customization_spec` | a customization specification from vCenter (**needed for Windows**) | `vm_deploy_template_specs`; Linux: none needed |
| `vcenter`, `datacenter` | which one, when the template exists in several | found by the job |
| `notes` | added to the VM's notes | - |

Other lines (a greeting, `Name: Jane Doe`) are ignored. Upper and lower case do not matter (`Memory: 16 GB` works too), and the names people use work as well: `Hostname:`, `Server Name:`, `Computer name:` (the VM's name; for several, `Hostnames:`, `Server names:` ... with the names after it, or one per line below it); `CPUs:`, `vCPU:`, `Cores:`, `Processors:`; `RAM:`, `Mem:`; `Disk:`, `Storage:`, `HDD:`, `Hard drive:`. A unit is read with the value: `Memory: 16384 MB` = 16 GB, `Storage: 100GB, 1TB` = 100 and 1024 GB, `Memory (GB): 16`.

**What wins** when two places give a value, from the least to the most:

| | Wins over the ones above it |
|---|---|
| 1 | the parent - the **change** a change task belongs to (its description) |
| 2 | the ticket's **short description** (plain words only) and **description** |
| 3 | the ticket's **work notes**, oldest first |
| 4 | the ticket's **comments**, oldest first |

So the **change task wins over its change**, and a **comment wins over a work note** (a requester's answer over a team member's note). A `key: value` line and plain words are ranked the same way: where it is written decides, not how - the task's *"named web02 with 6 vCPU"* wins over the change's `vm_name: web01` line. Two answers in the place that wins (*"4 vCPU, or maybe 8 vCPU"*) are a question, never an earlier place's value. `vm_deploy_parent_wins: true` moves the change above the task's short description and description (rows 1 and 2 swap); the task's work notes and comments still win.

## Plain words, without GenAI

With `vm_deploy_plain_words: true` the job also reads the ticket's own words, by rules you set, so a requester can write:

```
Hi, please build a Red Hat 9 server named ab01web05, medium size, for the new portal. Thanks, Jane
```

| What | Read from | Your setting |
|---|---|---|
| the template | **the operating system as people write it** - found from the template names in `vm_deploy_templates`, nothing to set: a template named *Windows 2022 Standard Template* is found by `Windows 2022`, `Windows Server 2022`, `Server 2022`, `Win2022`; *rhel9-gold* by `RHEL 9`, `RHEL 9.4`, `Red Hat 9`, `Red Hat Enterprise Linux 9` (also Rocky, Alma, CentOS, Oracle Linux, Ubuntu, Debian, SLES). Windows Server by its year alone: `Server 2025`, `a new 2025 VM`, `2022 servers` (not `SQL Server 2019`). An OS no template is for (`Server 2025`, only a 2022 template) is a question that names it - never another version. Two allowed templates for the same OS (a PXE and a Gold Windows 2022): a question. Also the words you list (`vm_deploy_plain_templates`), the template's own name and its short names (`vm_deploy_template_aliases`). Any case; `-`, `_` or nothing between the words; up to two other words between them | `vm_deploy_templates` with the templates' **exact names** (a pattern like `Windows 20*` has no OS to read: list the names, or use `vm_deploy_plain_templates`) |
| CPU and memory | a size word (`small`, `medium`, `large`), and numbers next to their unit: `4 vCPU`, `4 cores`, `CPU - 4`, `Processors: 4`, `2 sockets, 2 cores per socket` (= 4), `CPU : CORE.4`, `16 GB RAM`, `4 gigs of ram`, `RAM - 16GB`, `RAM : 16 GB.`, `Memory (GB): 16`, `16384 MB of memory` - a bare `16 GB` is not read | `vm_deploy_plain_sizes` |
| disks | `storage 180gb and 50gb`, `storage 180GB disk and 50gb disk`, `a 180GB disk and a 50GB disk`, `Disk 1: 100 GB` / `Disk 2: 200 GB`, `C: 100GB` / `D: 200GB`, `HDD'S C: /120`, `Disks: C: 100, D: 200` (a drive letter after a disk word: GB when no unit), `a 100GB OS disk and a 200GB data disk`, `hard drive 80 gigs`, `disks 1TB + 500gb` - disk 1 first (below) | - |
| the folder | `the Web-Servers folder`, `folder: Linux/New`, `in folder App`, `the "App Servers" folder` (a name with spaces in quotes) | - |

| What | Read from | Your setting |
|---|---|---|
| the VM's name | `named ab01web05`, `called ...`, `hostname: ...`, `server name ...`; several: `named web01 and web02`, `named web01, web02 and web03`, `2 VMs: web01 and web02` (how many, when said, must match the names), `spin up new 2025 VM / web01`, `new VM: web01`, `build server - web01` (each name written so is a new VM: two such lines, two VMs) - or else the one word that fits a name rule (`vm_deploy_name_rules`; longer than the rule's own letters, so a site code alone is not a name). Web and email addresses are skipped | the requester gives it |
| the site | a site's name (`vm_deploy_sites`) as a word of its own - any case, the spaces optional (`SiteTwo` = `Site Two`) - or the words you list for it; with one server per line, each line's own | `vm_deploy_plain_sites` |

YAML

```
vm_deploy_plain_words: true
vm_deploy_plain_templates:
  "rhel 9": rhel9-gold
  "red hat 9": rhel9-gold
  "windows 2022": WIN2022-TEMPLATE
vm_deploy_plain_sizes:
  small: {cpu: 2, memory_gb: 8}
  medium: {cpu: 4, memory_gb: 16}
  large: {cpu: 8, memory_gb: 32}
```

**Never a guess.** Two different answers in the text that says it last (`RHEL 9 or RHEL 8`, `small or medium`, two server names as in *"same as ab01web03, the new one ab01web06"*) are a **question**, written on the ticket as a **comment** (requesters see comments, not work notes): *"[AAP] We cannot build this VM yet: the ticket names more than one operating system ('rhel 9', 'rhel 8'): which one?"*. Nothing is built. The requester answers in a comment (*"RHEL 8 please"*): a later comment wins. Then run the job or the workflow again (the pickup does not take a ticket twice).

- **What wins:** `key: value` lines always; then the words; then a GenAI proposal; then the defaults. A size word and a number in the same text: the number (`medium with 32 GB RAM` = 4 vCPU, 32 GB). A later comment beats the description.
- **Every value says where it came from** in the job output, the propose note and the approver's email: `template: rhel9-gold (from the words 'red hat 9' in the description)`.
- **Your rules still apply:** the template the words lead to must be in `vm_deploy_templates`, CPU and memory within `vm_deploy_max_cpu` / `vm_deploy_max_memory_gb`.
- **Disks in words:** a disk word with sizes after it is a list, disk 1 first - `storage 180gb and 50gb` = disk 1 grown to 180 GB plus an added 50 GB disk; one size before a disk word is disk 1 (`180gb hard disk`, `a 100 GB system disk`, `C drive 100gb`), or an added disk when the words say so (`50gb second hard disk`, `a 200 GB data disk`, `D drive 200gb`); a numbered disk or a drive letter says which (`Disk 2: 200 GB`, `D: 200GB`). Two sizes for disk 1, or only added disks, are a question (a `disks_gb:` line answers it - or GenAI, when it is on).
- **CPUs:** sockets x cores per socket is the number of vCPUs (`2 sockets, 4 cores per socket` = 8); `2 sockets` alone = 2. Sockets and cores that do not add up (`2 sockets, 4 cores`: 4 or 8?) are a question.

- **The folder** the words name must exist in the datacenter (and be in `vm_deploy_folders`, when set): the dry run says when it does not.
- **With GenAI on too:** GenAI is asked when the words (and lines) do not give both the name and the template, **or the words left a question** (e.g. disks written in words, or two sizes) - and what the lines and words say wins over GenAI: GenAI fills only what is still open.
- A value is never read across a line break: `memory` at the end of one line and `180gb` on the next are not "180 GB of memory".

**A change laid out per server** - the servers only in its description, one line each - is read as it is:

```
1.  Build new web tier :  ab01web05 (front end)
a.  Platform team : spin up new 2025 VM / ab01web05 / CPU :  CORE.4  / RAM : 16 GB.  / HDD'S C:
  /120  Location: Site One
1.  Build new web tier :  ab01web06
a.  Platform team : spin up new 2025 VM / ab01web06 /  CPU :  CORE.4  / RAM : 16 GB.  / HDD'S C:
  /120 Location: Site Two
```

= two VMs (`ab01web05, ab01web06`), 4 vCPU, 16 GB, a 120 GB disk 1, Windows Server 2025; each at the site its own line names (`Site One`, `Site Two`: a site's name or its words, `vm_deploy_plain_sites`) - its vCenter and cluster from the site - else placed by its name rule.

- **Each line is its VM's own**: CPU, memory, disks, the operating system, the site. Lines that differ build VMs that differ (`ab01web05: 4; ab01web06: 8` in the dry run); a value only one line gives, or one written once for all (above the lines, or below them after a blank line), is for every VM - but never a site: a line with no Location takes none from another line.
- Two different values on one line (`CPU : CORE.4 / 8 cpu`) are a question about that VM only.
- A comment or work note with a line for all (`cpu: 8`) wins over every line (a comment over a work note).
- A name rule for a name used at several sites (`sites: [Site One, Site Two]`): a line with no Location, or one no site has, is a question - never the rule's vCenter.

Test it like any change: a dry run (**Check**) on a test ticket shows each value and where it came from.

## GenAI reads the ticket (optional)

- **Off by default** (`vm_deploy_genai: false`): nothing is sent to a model. The standard setup leaves it off; tickets written as `key: value` lines - or read by the plain-words rules - never need it.
- **On**, it is asked only when the ticket's own lines and words do not give both a name and a template (or left a question). It **copies values out** of the ticket - names, the OS as written, CPU, memory, disks - each with the words it came from. A requester can write *"We need a Windows Server 2022 box called winapp05 for SiteA, 8 GB RAM, 2 CPUs, a 120 GB system disk and a 200 GB data disk"*.
- **AAP decides everything:** it checks each quoted word is really in the ticket, picks the template from the OS words by its own rules, checks every limit, writes the proposal on the ticket, and builds. GenAI never sees a credential or vCenter, never chooses a template, a place or whether to build. A question it adds anyway is only a note in the job's report, never sent to the requester.
- **Propose, approve, build:** in the workflow's propose step, ACT reads the ticket (evidence mode: it runs no command, sees no credential and no inventory). The job posts what it read - `[AAP] GenAI proposal (ticket text sha256:...)`, each value with the words it came from - checks it, finds where it would go, and emails that to the approver. The build step builds **the posted proposal** - GenAI is not asked again - with every check of a normal build.

| Guardrail | What it does |
|---|---|
| Off by default | `vm_deploy_genai: false`; the `key: value` lines and plain words work without GenAI |
| A person approves | the workflow's approval sits between the proposal and the build; the build step builds the proposal posted on the ticket - never a fresh answer, never unseen (`vm_deploy_genai_approval: false` drops the person, for a demo) |
| The ticket must not change in between | the proposal carries a hash of the ticket text (the change's too); editing the description or adding a comment voids it: the build step builds nothing, and the workflow is run again from the start |
| Every value traceable to the ticket | each value comes with the ticket's words it is from; the job checks those words are in the ticket and support the value: the name word for word, the numbers as written, the **site named in its words**, the **template for the operating system its words (or, when they name none, the ticket) ask for** - anything else is dropped and becomes a question. GenAI's own confidence is not trusted |
| The template by the job's rules | the operating system GenAI copies gives the template by the plain words' rules; it may also name a key of `vm_deploy_template_catalog` (optional), which must be in `vm_deploy_templates` - a key for another version than the ticket asks for (2022 for *Server 2025*) is dropped. An OS no template is for is a question, never another version |
| Same rules as any request | allow-lists, CPU / memory / disk caps, name rules, the group, a free name: what the ticket text asks for ("use 64 CPUs") is refused like a typed request |
| No placement by GenAI | cluster and datastore come from the site, the defaults or the rules |
| Instructions in the ticket are data | the prompt tells the model to ignore them; the checks above make them harmless anyway |
| Everything on the record | the proposal, the questions, the "Deploying" note and the result are on the ticket; the job's artifacts keep what GenAI read |

**Set it up:** the deploy template gets the **ACT model key** credential (see ADDING_ACT.md for the provider and model), the approval workflow (*Set it up*, step 8), and in its Variables:

YAML

```yaml
vm_deploy_genai: true
vm_deploy_template_catalog:      # optional: when the template names do not say their OS
  win2022-pxe: "Windows Server 2022 (installed over the network by MECM)"
  rhel9-gold: "Red Hat Enterprise Linux 9"
```

- **Without the workflow** (a demo): launch the job template itself with `vm_deploy_genai_approval: false` - one run reads, checks and builds. With `vm_deploy_genai_approval: true` and no workflow, the job proposes and stops; launching it again builds the posted proposal.
- **A dry run (Check) with GenAI:** by default Check does not ask GenAI (it says *GenAI would read the ticket (dry run: not asked)*). With `vm_deploy_genai_in_check: true`, Check asks it too and shows its proposal - each value with the ticket's words, what was dropped, the questions it would send - and where the VM would go. It writes nothing on the ticket; a real run asks GenAI again.
- **The ticket text goes to the model** (GenAI.mil): only use it for tickets whose text may go there.

- `vm_deploy_genai_model_timeout` (600 s by default): the seconds the model may take.

# 5. Where the VM goes

## Who wins

For each VM, each placement value (`vcenter`, `datacenter`, `cluster`, `datastore`, `folder`) comes from the first of these that gives it:

1. **The ticket itself** - a `cluster:` (or `site:`, `folder:` ...) line, or the VM's own line in a change laid out one server per line.
2. **Its site** (`vm_deploy_sites`) - named by a `site:` line, the VM's own line (*Location: North Campus*), or the ticket's **Location** field. A site's name is found in any case, with or without spaces; `vm_deploy_plain_sites` adds the other ways people write the place.
3. **Its name rule** (`vm_deploy_name_rules`) - the first `match` that fits the VM's name. A rule may name a `site:` (step 2 then applies), or `sites: [SiteA, SiteB]` when the same name is used at two places: the ticket must then say which, never a guess.
4. `vm_deploy_defaults`**.**
5. **Automatic placement** among the candidates you list (a site's `clusters` / `datastores`, else `vm_deploy_clusters` / `vm_deploy_datastores`): the cluster with the most memory left with a host down, the least used datastore with room under 85 %. With `vm_deploy_datastore_balance: true`, a VM nothing else places goes on the least used shared datastore of its cluster (not the template's); the VMs of one ticket spread out.
6. **The template's own place.**

**Datastores by the VM's name** (`vm_deploy_datastore_rules`) come before steps 2 to 6: e.g. SQL servers on the SQL datastores, `[{match: ["*SQL*", "*DBSE*", "*DBV*"], datastores: ["*SQL*"]}]` - those VMs go on the least used of their rule's datastores, and no other VM goes there, even when it is the template's own. A ticket that names a datastore outside its rule is refused.

The allow-lists (`vm_deploy_clusters`, `_datastores`, `_folders`, when set) check a value from steps 1 to 4, and automatic placement chooses only among them. A site in another vCenter: that vCenter is in `vmware_vcenters`, and the template is there under the same name. The dry run shows each value's source: `cluster: A-CL01 (from site SiteA)`.

## Leaving things out: the defaults

A ticket need not say everything. `vm_deploy_defaults` fills in **any** value it leaves out - the template, CPU, memory and disks as well as the place - and what the ticket says still wins:

A ticket may say only the name

```
vm_deploy_defaults:
  template: rhel9-gold
  cpu: 2
  memory_gb: 8
  cluster: PROD-CL01
  datastore: DSC-PROD
  folder: Servers/New
```

| The ticket says | What is built |
|---|---|
| `vm_name: ab01web05` | rhel9-gold, 2 vCPU, 8 GB, in PROD-CL01 - each value shown as `(from default (vm_deploy_defaults))` |
| `vm_name: ab01web06`, `template: WIN2022-TEMPLATE`, `cpu: 4`, `memory_gb: 16` | the ticket's template and size, in PROD-CL01 |

- **With sites:** `vm_deploy_defaults: {site: SiteA}` sends a ticket that names no site to SiteA; a `site: SiteB` line (or its Location) still wins. Without a default site, a ticket that names none is refused - or, with `vm_deploy_site_required: false`, placed by the rest of the defaults.
- **A default template is still checked:** it must be in `vm_deploy_templates`, and a Windows one needs its Sysprep specification in `vm_deploy_template_specs` (unless it boots from the network).
- **Nothing for the place at all:** the job chooses among `vm_deploy_clusters` / `vm_deploy_datastores` (automatic placement) - and without those, uses the template's own cluster, datastore and folder (with `vm_deploy_datastore_balance: true`, the least used datastore of that cluster).

## Sites: site A goes to this cluster, site B to that one

YAML

```yaml
vmware_vcenters: [vc01.example.mil, vc02.example.mil]
vm_deploy_sites:
  SiteA: {vcenter: vc01.example.mil, cluster: A-CL01, datastore: A-DS01, folder: Servers/New}
  SiteB: {vcenter: vc02.example.mil, cluster: B-CL01, datastore: B-DS01, folder: Servers/New}
vm_deploy_plain_sites:              # other ways people write the place
  "north campus": SiteA
  "south campus": SiteB
```

The ticket says `site: SiteA` (upper / lower case does not matter) - or nothing, when the ticket's **Location** field is one of the site names (or their words). The site's values fill in what the ticket does not say (over `vm_deploy_defaults` and the name rules); a `cluster:` the ticket names itself still wins, within the allow-lists. With sites set up, a ticket that names no site is refused (`vm_deploy_site_required: false` uses `vm_deploy_defaults` instead). Any setting can go in a site: `cluster`, `datastore`, `folder`, `vcenter`, `datacenter`, `domain`, `cpu`, `memory_gb`.

- **The place as people write it.** With plain words on, a site's name in the ticket's text picks it (any case, the spaces optional: `Site Two`, `SiteTwo`, `site-two`) - and so do the words you list for it in `vm_deploy_plain_sites`, for every other way people write the place. The ticket's Location field is read through the same words.
- **One server per line:** a change that lists its servers one per line puts each VM at the site **its own line** names (`... / ab01web05 / ... Location: North Campus`).
- **A site in another vCenter** (`vcenter: vc02.example.mil`): that vCenter must be in `vmware_vcenters` (the job's vCenter credential works there too: one account for all, *Set it up*, step 2), and the template must be there under the same name. Give the site its own `cluster` (and `datacenter` when the names repeat across vCenters).
- **To test with one vCenter and one cluster**, point both sites at it; split them later by changing only `vm_deploy_sites`.

## Name rules: the VM's name picks the cluster

When your names say where a VM belongs (a site or room code in the name), let the name pick the cluster

- no `site:` or `cluster:` line needed in the ticket:

YAML

```
vm_deploy_name_rules:
  - {match: "*AB01*", cluster: YOUR-CLUSTER-FOR-AB01}            # every name with AB01 in it
  - {match: "*CD02*", cluster: YOUR-CLUSTER-FOR-CD02, folder: Servers/New}
  - {match: "??EF03*", site: SiteB}                              # 2 characters, then EF03: a
    site's settings
  - {match: "*XY05*", sites: [SiteA, SiteB]}                     # used at both: the ticket says
    which
```

- `match`: the name pattern - `*` = any characters, `?` = one character; upper and lower case alike. The **first** rule that fits the name is used.
- The rest of the rule fills in what the ticket does not say: `cluster` (or `clusters: [..]` to choose among), `datastore` (or `datastores: [..]`), `folder`, `vcenter` ... - or `site:` a name from `vm_deploy_sites`. A `cluster:` the ticket gives itself, or a site it names, still wins.
- With a name rule matched, the ticket needs no site (even with `vm_deploy_site_required`).
- **A name used at several sites** (the code in the name is the same at two places): `sites:` lists them, and the ticket must say which - by a site's name or its words (`vm_deploy_plain_sites`), on the VM's own line, or the ticket's Location field. Never a guess: with no site, or one not listed, it is a question - *"the ticket does not say where ab01xy05 goes - SiteA or SiteB: write it on its line, e.g. 'Location: SiteA'"*. Keep the placement (`vcenter`, `cluster` ...) in the sites, not in the rule.
- A rule's placement (`vcenter`, `datacenter`, `cluster`, `datastore`, `folder`) is never used for a VM that its site, or the ticket, puts in another vCenter.
- The cluster must still be allowed (`vm_deploy_clusters`, when you set that list).
- Datastore: the rule's, else the site's or `vm_deploy_datastores` candidates, else the template's - and when the template's datastore is not on that cluster, the cluster's own least used datastore (shared, under 85 % with the VM). `vm_deploy_datastore_rules` (SQL servers) wins over all of them.
- The dry run (Check) shows it: `cluster: YOUR-CLUSTER-FOR-AB01 (from name rule *AB01*)`.
- The settings finder (*Set it up*, step 5) writes one rule per cluster for you to fill in.

## Automatic placement: the job picks the cluster and datastore

When neither the ticket, its site nor `vm_deploy_defaults` names a cluster (or a datastore), the job chooses one by rules - never from all of vCenter, only among explicit candidates:

- **Candidates:** a site's `clusters` / `datastores` lists, else `vm_deploy_clusters` / `vm_deploy_datastores`. With neither, the VM goes where the template is.
- **Cluster:** the one with the most memory left after this VM, planning one host down (`vm_deploy_failover_hosts`) - it must stay under `vm_deploy_mem_target_pct` (80 %).
- **Datastore:** shared, accessible, not in maintenance, mounted by that cluster, and under `vm_deploy_datastore_max_used_pct` (85 %) with the VM at its **full** size (thin disks counted full). The **least used first**: the one that is the least full with the VM on it. The VMs of one ticket spread out - each counts what the ones before it in the same run put on a datastore.

- The ticket note and the report give the ranking: `cluster chosen: PROD-A2: memory 61.3% used with this VM and 1 host down (target 80%)`, `datastore chosen: A-DS-03: 41.2% used with this VM (at most 85%)`.

```yaml
vm_deploy_sites:
  SiteA: {clusters: [A-PROD-*], datastores: [A-DS-*], folder: Servers/New}
  SiteB: {clusters: [B-PROD-*], datastores: [B-DS-*], folder: Servers/New, vcenter:
    vc02.example.mil}
```

**By location, the least used first - for every VM.** The location picks the cluster (a site, a name rule, the ticket); the datastore is then the least used of that cluster's. Without datastore lists, switch on `vm_deploy_datastore_balance: true`: a VM nothing else places goes on the least used shared datastore of its cluster (instead of the template's own). A site's or a rule's single `datastore:` still puts it there.

**SQL servers on their own datastores** (`vm_deploy_datastore_rules`): the VM's name picks its datastores.

```yaml
vm_deploy_datastore_rules:
  - {match: ["*SQL*", "*DBSE*", "*DBV*"], datastores: ["*SQL*"]}   # e.g. ab01sql01, AB01DBSE02,
    ab01dbv03
vm_deploy_datastore_balance: true                                    # every other VM: the least
  used of its cluster's
```

- The **first** rule whose `match` fits the name (`*` and `?`, any case) gives the datastores the VM goes on: the least used of them on its cluster. That wins over a site's, a name rule's or `vm_deploy_defaults`' datastore; a `datastore:` the ticket writes itself must be one of them (else a question: *"ab01sql01 goes on a datastore *SQL\ ...: A-DS01 is not one"*).
- A VM **no rule matches never goes on a datastore a rule names** - they are kept for those VMs, even when it is the template's own (then: the least used of the cluster's others).
- None of them on the VM's cluster, or none with room: nothing is built - *"no datastore among *SQL* of A-PROD-CL01 has room for 140 GB"*. The check in vCenter before the build finds that, for every VM.

## Empty templates: network boot to MECM, or boot from a CD (kickstart)

A template with **no operating system** that installs it itself - it boots from the network into an MECM (SCCM) task sequence, or from its CD / ISO into a kickstart: list it in `vm_deploy_pxe_templates` (e.g. `["WIN20*-PXE", "Rhel*-KS"]`). For those, the job:

- names the VM in vCenter as the ticket says (`vm_name`), like every deploy;
- does **no guest customization** (there is no OS yet) and does not wait for an IP address;
- refuses, for a Windows guest, a `vm_name` longer than 15 characters (the Windows / MECM computer name limit);
- gives the VM the ticket's name and number as VM settings the installer can read - `guestinfo.hostname` and `guestinfo.ticket` (`vm_deploy_guestinfo`; vCenter privilege *Change configuration* > *Advanced configuration* - without it the VM is still built and a note on the ticket says the name was not set there);
- powers the VM on - it boots to MECM or its CD - and writes on the ticket its **MAC address** and the computer name to use.

**The name inside the OS** comes from the installer, not from vCenter:

- **A kickstart (Linux, from the CD):** read the name in `%pre` and use it, for example

```
%pre
name=$(vmtoolsd --cmd "info-get guestinfo.hostname" 2>/dev/null)
[ -n "$name" ] && echo "network --hostname=$name" > /tmp/hostname.ks
%end
%include /tmp/hostname.ks
```

This needs VMware Tools in the installer (`vmtoolsd`, from open-vm-tools); check your ISO (boot it, press Ctrl+Alt+F2 and type `which vmtoolsd`). Without it: a DHCP reservation for the MAC address on the ticket gives the name, or whoever finishes the install sets the name from the ticket.

- **Windows (MECM, from the network):** the name comes from MECM, not from `guestinfo` - Windows PE has no VMware Tools to read it. Use one of the two MECM ways below.

On the MECM side, one of the two usual ways:

1. **Unknown computers:** the task sequence is deployed to *All Unknown Computers* and asks for (or sets) the computer name - use the VM's name from the ticket.
2. **Known computers:** an MECM admin imports the computer (**Assets and Compliance > Devices > Import Computer Information**: name = `vm_name`, MAC = the one on the ticket) into the collection the task sequence is deployed to.

The task sequence deployment is **Required**, made available to **Only media and PXE**, and deployed to a collection that holds only the new VMs (the imported computers, or *All Unknown Computers*) - never to one with servers in use: a Required deployment that also goes to *Configuration Manager clients* runs on every client in its collection and installs it again. An **Available** deployment waits at the VM's console for someone to press a key: F12 on a BIOS VM, Enter (*Press ENTER for network boot service*) on an EFI one. After it succeeded, MECM does not start it again on that computer; to rebuild the same VM: the device in MECM > **Clear Required PXE Deployments**.

**The template, for a network boot:**

- **Boot order: leave VMware's default, the hard disk first and the network after it.** The empty disk is skipped, so the first boot goes to the network (PXE) and MECM. Once the task sequence has installed Windows, the disk can boot and starts first (with EFI, Windows Setup also puts *Windows Boot Manager* first in the firmware). Nothing to change after the install. With the network first, every restart would ask MECM again.
- **VM Options > Boot Options > Failed Boot Recovery:** on, 10 seconds - a network boot MECM did not answer (e.g. the computer is not imported yet) is tried again instead of stopping at "no operating system".
- **Firmware EFI** (Secure Boot may stay on: MECM's boot images are signed by Microsoft).
- **The adapter:** on the staging VLAN, *Connect At Power On* ticked, and the VLAN forwards PXE to your MECM distribution point (DHCP / IP helper). A **VMXNET3** adapter needs VMware's vmxnet3 driver in the MECM **boot image** (Windows PE has none: it starts, then the task sequence finds no network) - MECM: import the driver (from VMware Tools, `Drivers\vmxnet3`), add it on the boot image's **Drivers** tab and update the distribution points. An **E1000E** adapter works without.
- **The disk:** empty, never initialized.

**The template, for a CD boot:** the CD drive points at the ISO on a **datastore every target cluster's hosts can see** (a clone in a cluster that cannot see it boots to nothing), with *Connect At Power On* ticked, and the **hard disk before the CD** in the boot order (VMware's default): the empty disk is skipped, so the first boot starts the ISO, and once installed the VM starts from its disk. With the CD first it can start the installer again,

unless the kickstart ends with `reboot --eject`. The job never attaches or chooses an ISO: the template's CD drive carries it.

**Closing the ticket:** a network-boot VM is "built" when it is powered on and booting - the OS install is not finished yet. To leave those tickets open for whoever checks the install: `vm_deploy_done_fields_pxe: false`.

# 6. Ready-made configurations

Each block is a complete starting point for the deploy template's Variables. Take one, change the example names to yours, run a dry run (Check) on a test ticket.

**1. The smallest: `key: value` tickets, one cluster.**

One cluster, tickets written as key: value lines

```
vm_deploy_templates: [rhel9-gold, WIN2022-TEMPLATE]       # REQUIRED: nothing else is deployed
vm_deploy_template_specs: {"WIN2022*": Win2022-Sysprep}   # Windows: its Sysprep specification
vm_deploy_assignment_group: Server Team
vm_deploy_defaults: {cluster: PROD-CL01, datastore: DSC-PROD, folder: Servers/New}
vm_deploy_max_cpu: 16
vm_deploy_max_memory_gb: 128
vm_deploy_domain: example.mil
report_email_to: vmteam@example.mil
```

**2. Launched with the change, any group's; the team's task gets the notes and is closed by hand.**

Launched with the change - the task closed by hand

```
vm_deploy_templates: [rhel9-gold, WIN2022-TEMPLATE]
vm_deploy_assignment_group: Server Team          # finds the team's task under the change
vm_deploy_assignment_group_by_table:             # the change itself may be any group's
  change_request: ""
vm_deploy_notes_also: [parent, tasks]            # every note on the change and the task
vm_deploy_close_tasks: false                     # the team closes the task
vm_deploy_plain_words: true                      # "a Windows 2022 server named ab01web05"
vm_deploy_defaults: {cluster: PROD-CL01, datastore: DSC-PROD, folder: Servers/New}
report_email_to: vmteam@example.mil
```

**3. The same, the task closed when every VM is built.** Configuration 2, with:

...and the task closed when complete

```
vm_deploy_close_tasks: true
vm_deploy_done_fields_by_table:
  change_task: {state: "3", close_code: successful, close_notes: "VMs deployed by AAP"}
vm_deploy_done_fields_issues_by_table:
  change_task: {close_code: successful_issues}
vm_deploy_done_fields_pxe: false     # MECM (network-boot) VMs: left open for the install check
```

**4. Two sites in two vCenters; the name and the Location pick the site.**

Sites, the words for them, and name rules

```
vmware_vcenters: [vc01.example.mil, vc02.example.mil]
vm_deploy_sites:
  SiteA: {vcenter: vc01.example.mil, cluster: A-CL01, datastore: A-DS01, folder: Servers/New}
  SiteB: {vcenter: vc02.example.mil, cluster: B-CL01, datastore: B-DS01, folder: Servers/New}
vm_deploy_plain_sites:              # other ways people write the place
  "north campus": SiteA
  "south campus": SiteB
vm_deploy_name_rules:
  - {match: "*AB01*", site: SiteA}            # every name with AB01 in it
  - {match: "*XY05*", sites: [SiteA, SiteB]}  # used at both: the ticket's Location says which
```

To test with one vCenter and one cluster, point both sites at it; split them later by changing only `vm_deploy_sites`.

**5. Network-boot templates (MECM installs Windows).**

Empty templates that boot from the network

```
vm_deploy_templates: [WIN2022-PXE, rhel9-gold]
vm_deploy_pxe_templates: ["WIN20*-PXE"]   # no customization; the MAC address goes on the ticket
vm_deploy_name_regex: '^[A-Za-z0-9-]{1,15}$'   # Windows computer names: 15 characters at most
```

**6. GenAI reads plain-words requests; a person approves before the build.** The workflow `VM - deploy from ServiceNow ticket (workflow)`: propose step, approval, build step (step 8 of *Set it up*), and the ACT model key credential on the deploy template.

GenAI with an approval

```
vm_deploy_plain_words: true
vm_deploy_genai: true
vm_deploy_genai_approval: true      # false = one run reads, checks and builds (a demo)
vm_deploy_genai_in_check: true      # a dry run shows GenAI's proposal too (writes nothing)
```

**7. The schedule picks up the change tasks** - on the pickup template (`playbooks/vm_deploy_poll.yml`), not the deploy one:

The pickup template: change tasks whose change is approved and in Implement

```
vm_deploy_assignment_group: Server Team
vm_deploy_poll_tables: [change_task]
vm_deploy_poll_prefix: "VM request"     # the tasks' short description starts with it
vm_deploy_poll_query: "change_request.approval=approved^change_request.state=-1"
vm_deploy_poll_states: ["1"]
vm_deploy_poll_dry_run: true        # the first week: list only
```

**8. The server's service: words in the VM's notes, an AAP inventory and group** - with the *Service* survey question and a Red Hat Ansible Automation Platform credential whose user has Admin on those inventories (step 10 of *Set it up*):

The server's service, picked at launch

```
vm_deploy_services:
  sql: {inventory: Windows servers, group: win_sql, note: SQL Server}
  web: {inventory: Linux servers, group: web}
  test: {note: Test server - delete after 30 days}
vm_deploy_inventory_name: "{name}.example.mil"
```

# 7. When something goes wrong

Every refusal says what is wrong and builds nothing; the ticket gets the same words as a note (a work note - or a comment, for a question to the requester). Find the message below. The job's output starts each step with `VMware |`; the line that failed names the step.

## The deploy job

### Launching it, reading the ticket

| You see | It means | Do |
|---|---|---|
| `No ServiceNow connection: attach the "ServiceNow API" credential` | the template has no ServiceNow credential | add it (*Set it up*, step 6) |
| `Give the ticket number: the job got none` | the template has no survey, or it is switched off | the Survey tab: *Ticket number*, variable `vm_deploy_ticket`, text, required - and switch it on |
| `Which table is INCN0012345 in? ...` | your instance numbers its tickets its own way, and the account may not read the task table (or there is no such ticket) | `vm_deploy_ticket_prefixes: {INCN: incident, CHGN: change_request}` on the template; check the number |
| `ServiceNow has no table "..."` (before: `HTTP Error 400: Bad Request`, `Invalid table change`) | `vm_deploy_ticket_table` or `vm_deploy_ticket_prefixes` names a table ServiceNow does not have | leave both empty: the job finds each ticket's table. `change`, `change request`, `change task`, `catalog task` are read as `change_request`, `change_task`, `sc_task` |
| `No change_request ticket INCN... in ServiceNow. vm_deploy_ticket_table ... is set` | `vm_deploy_ticket_table` sends every number with a prefix the job does not know to one table, and this ticket is in another | leave it empty (a `CTASK...` or `INC...` number is never sent there) |
| `No incident ticket INC... in ServiceNow` | a wrong number, or the account cannot see it | check the number; the account needs read access to that table |
| `ServiceNow refused the read (HTTP 403)` | the account may not read that table | give it read access (e.g. the `itil` role) |
| `vCenter ... refused the login` | no domain in the user name (a vCenter account is `name@vsphere.local`), a wrong password, a locked account - or that vCenter does not know the account | fix the **VMware vCenter** credential; log in to the vSphere Client of **that** vCenter with the same account to test it. The deploy stops when any vCenter of `vmware_vcenters` refuses: it checks the new name is free in all of them |
| `vCenter ... accepted the login, but the account has no permission in that vCenter` | the account has no role anywhere in that vCenter | a **Global Permission** (*Set it up*, step 2); read-only is enough on a vCenter the job only reads |

### The change, the group

| You see | It means | Do |
|---|---|---|
| `the change CHG... (this ticket's parent) is not ready to build: approval is 'Requested', not Approved; state is 'Assess', not Implement` | the change is not approved, or not in Implement (yet, or any more) - nothing was built | approve it / move it to Implement and run the job again; your instance's labels differ (e.g. *Implementation*): put them in `vm_deploy_ready_by_table` |
| `this ticket (CHG...) is not ready to build: ...` | launched with a change that is not approved and in Implement | the same |
| `... approval is empty, not Approved` | the account may not read the change's approval field, or the table has none | read access to `change_request.approval`; or take `approval` out of `vm_deploy_ready_by_table` |
| `REFUSED: the ticket is assigned to '...', not to ... (vm_deploy_assignment_group)` | not your team's ticket | a change of another group: `vm_deploy_assignment_group_by_table: {change_request: ""}`; a task: assign it to your group, or correct `vm_deploy_assignment_group` |
| `No open task of CHG... matches ...: the notes are on the change only.` | the task is not assigned to your group, is closed, or its words do not match `vm_deploy_notes_tasks_query` | copy the query from the change task list's filter (*Copy query*) |

## The request

| You see | It means | Do |
|---|---|---|
| A change task gets questions ("which operating system ...") though its change says it all | plain words are off, or the request is in another field of the change | `vm_deploy_plain_words: true`; `vm_deploy_parent_text_fields: [description, implementation_plan]` |
| `the ticket does not say vm_name` | no name the job can read | the requester adds a comment `vm_name: ...` (or *named* ... with plain words) |
| `the ticket asks for 2 VMs ('2 vms', ...) but names 1 (web01)` | the words say how many VMs, and the names found are fewer (or more) | the requester writes every name (`named web01 and web02`) in a comment, or a line `vm_name: web01, web02` |
| `the ticket gives a range of names ('named web01 to web05' ...)` | a range is never expanded | the requester writes every name; a line `vm_name: web01-05` means one VM of that name |
| `the ticket asks for 12 VMs (...): more than 10 (vm_deploy_max_vms)` | more names than one ticket may build | split the ticket, or raise `vm_deploy_max_vms` on the template |
| `for ab01web06: the ticket gives more than one number of CPUs ...` | that VM's own line gives two values (one server per line) | the requester corrects that line, or writes a line for all (`cpu: 4`) |
| `REFUSED: template '...' is not in vm_deploy_templates` | a template you have not allowed | add it to `vm_deploy_templates`, or correct the ticket |
| `the ticket asks for the operating system Windows Server 2025 (...), and no template here is for it` | no allowed template's name (or `vm_deploy_plain_templates` words) is for that version - the job never builds another one | add a template for it to `vm_deploy_templates` (and `vm_deploy_pxe_templates` when MECM installs it); or, when an existing one is right for it (an empty MECM shell): `vm_deploy_plain_templates: {"server 2025": "<template>"}`; or `template: <name>` on the ticket |
| `template ... is in vc01 / DC1, vc02 / DC1` | the same template name in two places | `vcenter:` (or `datacenter:`) in the ticket, a site, or a name rule |
| `template ... is Windows and has no customization specification` | an empty template that installs Windows from the network (MECM) is not listed as one - or a template with Windows installed has no Sysprep specification | the empty one: add it to `vm_deploy_pxe_templates` (no customization then); one with Windows: `vm_deploy_template_specs: {"Windows 2022*": <spec>}` - the message lists the vCenter's Windows specifications - or a ticket's `customization_spec:` line |
| `disk 1 cannot shrink` | `disks_gb` asks for a smaller first disk than the template's | the first size at least the template's |

### The service and its AAP inventory

| You see | It means | Do |
|---|---|---|
| `service webb is not one of vm_deploy_services (...)` | the *Service* answer is not a name in `vm_deploy_services` - nothing was read, built or written on the ticket | the survey's choices must be exactly those names; launch again |
| `service sql (vm_deploy_services): inventory Windows servers has no group win_sql: create it in AAP` (or `no inventory ...`, `... is a constructed inventory, may not change inventory ...`) | the service's inventory or group is not in AAP as written, or the AAP user lacks the Admin role there - nothing was built | create the group (the job never does), fix the name in `vm_deploy_services`, or give the credential's user the **Admin** role on the inventory |
| `2 inventories are named Linux servers (organizations ...): name it by its ID` | the same inventory name in two organizations | its ID in `vm_deploy_services` (`inventory: 42`, the number in its address in AAP) |
| `No AAP connection: attach a credential of type "Red Hat Ansible Automation Platform"` | a service with an inventory, and no AAP credential on the deploy template | attach one whose user has Admin on the inventory (*Set it up*, step 10) |
| `the CA file ... (vm_deploy_aap_ca_path) is not there` | the CA file for AAP's certificate is not where the setting says | the path as the EE sees it, or leave the setting empty |
| `WARNING: AAP inventory ...: NOT added - ... Add app01 by hand` | the VM is built; AAP did not take the host (a role, AAP down) | add it by hand, or fix the cause and run the job again: the VM is left as it is and the host is added |
| `WARNING: inventory ... is filled by ... with Overwrite on` | that inventory's source may take the new host out of its groups at its next sync | untick **Overwrite** on the source, or add the server to the source's file as well |

## Where it goes

| You see | It means | Do |
|---|---|---|
| `the ticket does not say which site` | sites are set up, the ticket names none, its Location is not a site, and no name rule fits the name | `site: ...` in a comment; a Location the sites know (`vm_deploy_plain_sites`); a name rule; or `vm_deploy_site_required: false` |
| `site '...' is not one of vm_deploy_sites` | a site name you have not set up | correct the ticket, or add the site |
| `the ticket does not say where ab01xy05 goes - SiteA or SiteB` | its name rule has `sites:` (a name used at several sites) and its line names no site - or a place no site's name or words match (`Location: N. Campus`, the site's words say `north campus`) | the requester writes the site on the VM's line; or add the way they write it to `vm_deploy_plain_sites` |
| `ab01xy05 is to go to site SiteC, but its name rule *XY05* is for SiteA or SiteB` | the VM's line (or a `site:` line) names a site its name rule is not for | correct the ticket - or add the site to the rule's `sites` |
| The VM went to the default (or the template's) cluster, not the name rule's | no rule's `match` fits the name (a typo, or `AB01*` when the name starts with other letters) | `*AB01*` (anywhere in the name); check the dry run's `(from name rule ...)` |
| `no cluster named X in datacenter Y: X is a vCenter, not a cluster - the clusters there: ...` (or *an ESXi host*, *the datacenter*) | the cluster setting names something else: often the vCenter's name in `cluster:` | the job's output shows where it came from (`cluster: X (from ...)`); put one of the clusters listed there, or no cluster at all - the template's own. A name written in other case is found (with a note) |
| `no cluster among ... has room` | automatic placement: every candidate would go over the memory target with a host down | another site / cluster, less memory, or a higher `vm_deploy_mem_target_pct` |
| `no datastore among ... has room` | every candidate datastore would go over 85 % with the VM | another datastore list, smaller disks |
| `goes on a datastore *SQL* ...: A-DS01 is not one` / `is kept for other VMs` | `vm_deploy_datastore_rules`: the ticket names a datastore the VM's rule does not allow, or one kept for other VMs | drop the `datastore:` line (the job picks the least used of the allowed ones) |
| `no VM folder Test/App in datacenter DC1 (a path is looked up as it is ...)` / `is not one of vm_deploy_folders` | a folder path is that path exactly (never another folder of the same name); the folder found must be allowed | write the path as in vCenter, below the VM folder (the settings finder lists them) |

## Building it in vCenter

| You see | It means | Do |
|---|---|---|
| `Nothing was built: ServiceNow refused the work note` | the account cannot update that table | give it write access to work notes; nothing was built, by design |
| `a VM named ... already exists in vCenter ...` | the name is taken by a VM this ticket did not build (its notes do not carry the ticket) | another name in a comment - or nothing to do, when it is the right VM |
| `a VM named web01 already exists` on a ticket that built it | the VM's notes no longer name the ticket (edited), or the ticket has no "deployed" note for it | check that it is the right VM; delete it to build again, or close the ticket by hand |
| `an earlier run of this ticket built it, and its guest customization failed` | its notes' first line says *failed* | check the VM's events in vCenter, delete it, run the job again |
| `Nothing was built: ... 1 of 3 cannot be: - web03: ...` | several VMs: one fails the check in vCenter (name taken, no template, no datastore with room ...) - none is built | fix that one (or the ticket), run again |
| `lacks the privilege VirtualMachine.Provisioning...` (or another) | the vCenter account's role | grant it (*Set it up*, step 2) |
| `lacks the privilege System.View on host-...` | the account's role is on the cluster without **Propagate to children**, or the host is in a cluster it has no role on | the role on those clusters (and the template's) with **Propagate to children**, or a Global Permission. A host's name for an ID: `https://<vcenter>/mob/?moid=host-...` |
| `lacks the privilege Cryptographer.Clone ... the template is encrypted` | the template has a vTPM (or encrypted disks) | the *Cryptographic operations* privileges (*Set it up*, step 2) |
| Customization `no result within 900 s` | VMware Tools does not run in the template, or cloud-init blocks VMware customization | fix the template; the VM exists - set its host name by hand, or delete it and run again |
| `no IPv4 address within 600 s` | no DHCP lease on the staging VLAN, or VMware Tools does not run | check DHCP; the IP shows in vCenter when it comes; the VM exists |
| `the template's ... is not set to connect at power on` | the template's network adapter | tick **Connect at power on** on the template |
| A network-boot VM waits at *Press ENTER for network boot service* (F12 on a BIOS VM) | the MECM task sequence deployment is **Available**: it waits for a key | make it **Required**, available to **Only media and PXE**, on a collection of only the new VMs (*Empty templates*, "On the MECM side") |

| You see | It means | Do |
|---|---|---|
| A network-boot VM sits at "PXE-E..." / "No boot device" | the VLAN does not reach MECM's PXE, or MECM does not answer this computer | the DHCP / IP helper on the staging VLAN; the computer imported in MECM, or unknown-computer support on the task sequence |
| `[AAP] 1 of 2 VMs built (...); this ticket stays open` | one VM failed (its own note says why), the others were built | fix the cause and run the job again: the VMs built are left as they are, the rest are built, the ticket is closed |

## Closing the ticket

| You see | It means | Do |
|---|---|---|
| `vm_deploy_close_tasks is on, but vm_deploy_done_fields_by_table has no change_task entry` | closing is on, but nothing says how | add the `change_task` entry (*Closing the task*) - or switch closing off |
| `NOT closed - close it by hand: ...` | ServiceNow refused the close; its reason follows | add the mandatory field to the close values; values from *Show XML* |
| The task closed, but its *Close code* shows a value in grey / not in the list | a label was sent, or a value your instance does not have | take the values from a closed task's *Show XML* |

### The workflow and GenAI

| You see | It means | Do |
|---|---|---|
| `... ready to build in ... - waiting for the approval` | the propose step (or the job alone with a GenAI proposal): by design | approve the workflow (or launch the job again) |
| `was changed after the approval` / `has no "[AAP] Checked - ready to build" note` | the build step: the ticket (or its change) changed after the propose step checked it, or that step ran with an older release | run the workflow from the start |
| `has no GenAI proposal for its text as it is now` | the build step, but the ticket changed after the proposal (or there is none) | run the workflow from the start |
| `questions for the requester (nothing built)` | the job could not trace a value GenAI gave to the ticket, or something it needs is missing (GenAI's own questions are only `GENAI NOTE` lines, never sent) | the requester answers in a comment (`memory_gb: 16`); run the job again |
| `DROPPED: vm_name ...` | GenAI gave a value the ticket does not contain (made up, or guessed) | nothing: it is a question now |
| `DROPPED: template '...': it is for Windows Server 2022, the ticket asks for Windows Server 2025` | GenAI named a catalog template for another version | nothing: never built (*The request*, above) |
| `GenAI did not run: ...` | no ACT model key, the model not reachable, or a timeout | the ACT model key credential on the template; ADDING_ACT.md; or `key: value` lines |
| `GenAI did not read the ticket: GenAI gave no readable answer` | the model answered, but not with the JSON the job asks for (a chat reply, a refusal, an answer cut short) | the output shows what it answered under **WHAT GENAI ANSWERED**: a chat reply means the model ignored the task - try another model (`site_act_model`); cut short - a longer `vm_deploy_genai_model_timeout`. Plain words work without GenAI |

## The scheduled pickup

| Message | What to do |
|---|---|
| `attach a credential of type "Red Hat Ansible Automation Platform"` | *Set it up*, step 9.3, on the pickup template |
| `AAP refused the login (HTTP 401)` | the credential's user name and password (or token) |
| `no workflow named "..." that the AAP user may see` | the workflow's exact name in `vm_deploy_poll_workflow`, and the Execute role (step 9.2) |
| `takes no ticket number` | the workflow's survey question `vm_deploy_ticket` (or Prompt on launch for Variables) |
| `AAP did not answer at ...` | AAP's address in the credential; its certificate (`vm_deploy_poll_aap_ca_path`) |
| `vm_deploy_assignment_group is empty` / `nothing marks a VM request` | set the group, and the prefix or the query |
| `STOPPED - this account cannot see the work notes it writes` | the ServiceNow account needs to read work notes (the `itil` role); or set `vm_deploy_poll_claim_fields: {state: "2"}` so a picked-up ticket leaves the New state. Until then `vm_deploy_poll_enabled: false` |
| `NOT STARTED: ... AAP did not start it` | the reason is on the ticket too; fix the workflow (its templates, inventory, credentials), then start it by hand with that ticket number - the pickup does not try a ticket twice |
| a ticket is never picked up | the job's output lists every ticket it read and why it skipped it; a ticket not listed at all did not match the query (the `Query:` line): its group, its state, its short description |
| `only the oldest 100 were read` | tickets AAP already has stay New and count too: set `vm_deploy_poll_claim_fields: {state: "2"}`, or resolve them |

# 8. Where the logic lives

| File | What is in it |
|---|---|
| `playbooks/vm_deploy_from_ticket.yml` | the playbook the template runs: it only includes the role's `deploy.yml` |
| `roles/vmware_vm/defaults/main.yml` | every `vm_deploy_*` setting, its default and what it does |
| `roles/vmware_vm/tasks/deploy.yml` | reads the ticket and its change; finds the change's task (*"the change's tasks that get the notes too"*); which step this is (propose, build); is GenAI needed |
| `roles/vmware_vm/tasks/deploy_service.yml` | the service picked at launch: its words, its AAP inventory and group checked - before the ticket is read |
| `roles/vmware_vm/tasks/deploy_genai.yml` | the GenAI step: the model copies values out, each checked against the ticket |
| `roles/vmware_vm/tasks/deploy_build.yml` | the rules check, every VM checked in vCenter first, the *Deploying* note (what an earlier run left), the loop over the VMs, *"close the ticket"* and *"close the change's tasks"*, the report and its email |
| `roles/vmware_vm/tasks/deploy_build_one.yml` | one VM: built (or left / finished, from an earlier run), joins its service's AAP inventory, then its result note at once |
| `roles/vmware_vm/tasks/deploy_register.yml` | one VM joins its service's AAP inventory and group - a failure is a warning, never a stop |
| `roles/vmware_vm/tasks/deploy_note_also.yml` | the same note on the change, or on its task - notes only, never a close |
| `roles/vmware_vm/library/site_vmware_deploy.py` | the vCenter work: the ticket's mark on a VM (built, started - picked up), the template, automatic placement (least used, the datastore rules), clone, CPU, memory, disks, customization or network boot, power on, the IP address |
| `roles/vmware_vm/library/site_aap_inventory.py` | AAP's API: an inventory and group checked (never created), a host added and put in its group (looked up first, never twice) |
| `plugins/filter/vm_deploy_filters.py` | the decisions, in plain Python - below |
| `playbooks/vm_deploy_poll.yml`, `roles/vmware_vm/tasks/deploy_poll.yml` | the scheduled pickup |
| `playbooks/vm_deploy_inventory.yml`, `roles/vmware_vm/tasks/deploy_inventory.yml` | the settings finder |

| File | What is in it |
|---|---|
| aap/extra_vars/vm_deploy_from_ticket.yml | every setting, ready to paste into the template's Variables |
| docs/vm_deploy_examples.yml | an example for every setting, and when to set it (the last chapter of the PDF) |
| tests/test_vm_deploy.py, tests/vmware/run_vmware_test.sh | the unit tests; whole runs against a vCenter simulator, a ServiceNow stand-in and an AAP stand-in (tests/servicenow/aapmock.py) |

The decisions in plugins/filter/vm_deploy_filters.py, by function:

| Function | Decides |
|---|---|
| vm_deploy_spec | what the ticket asks for, and who wins: the place it is written (change < description < work notes < comments), then a GenAI proposal, then the defaults |
| vm_deploy_words | the plain words: templates, sizes, sites, one line per server |
| vm_deploy_check | the rules: templates, caps, names, the group, sites and name rules, the datastore rules, approved and in Implement |
| vm_deploy_checked_digest | the ticket text the approval was for (the build step builds only that) |
| vm_deploy_ready | is the change approved and in Implement (vm_deploy_ready_by_table) |
| vm_deploy_group_for | which group the ticket launched must be (vm_deploy_assignment_group[_by_table]) |
| vm_deploy_tasks_query | the ServiceNow query for the change's tasks that get the notes |
| vm_deploy_note_also | which other tickets get each note |
| vm_deploy_close_result | did ServiceNow really close it (the values read back) |
| vm_deploy_genai_task, vm_deploy_genai_parse | what GenAI is asked; its answer checked word by word against the ticket |
| vm_deploy_built | the VMs an earlier run built for this ticket (never built twice) |
| vm_deploy_service_pick | the service picked at launch: its words, its inventory and group (vm_deploy_services) |
| vm_deploy_inventory_host | the VM's name in the AAP inventory (vm_deploy_inventory_name) |
| vm_deploy_inventory_text | what the ticket and the report say about it (*added as …*, *NOT added …*) |

# 9. Every VM deploy setting: its default, an example, when to set it

Every `vm_deploy_*` setting of the deploy job and of the scheduled pickup, in the order of `roles/vmware_vm/defaults/main.yml`: the value it has when you set nothing, the whole comment the file has for it, an example (`docs/vm_deploy_examples.yml`) and when you would set it. The other VMware settings (`vmware_vcenters`, the vCenter connection, the report email) are in the *Settings Reference* PDF.

Each one is marked:

- **Standard setup** - in *Start here: the standard setup* (chapter 1): `vm_deploy_ticket`, `vm_deploy_notes_also`, `vm_deploy_notes_tasks_query`, `vm_deploy_close_tasks`, `vm_deploy_templates`, `vm_deploy_assignment_group`, `vm_deploy_defaults`, `vm_deploy_template_specs`, `vm_deploy_done_fields_by_table`, `vm_deploy_done_fields_issues_by_table`.
- **Set when needed** - when the *When* line describes your site.
- **Leave as is** - the default is right unless you have that case.

## Deploy a VM from a ServiceNow ticket (playbooks/vm_deploy_from_ticket.yml)

| Setting | What it does - an example - when to set it |
|---|---|
| `vm_deploy_ticket`<br>Default: (empty)<br>**Standard setup** | The ticket (survey answer): INC..., TASK..., SCTASK..., RITM..., CHG... - or your instance's own prefixes (INCN..., CHGN...): the job asks ServiceNow which table a ticket is in. The ticket says what to build in 'key: value' lines - in its description, or in a comment / work note (a newer one overrides): vm_name: app01 template: rhel9-gold cpu: 4 memory_gb: 16 (cluster, datastore, folder, datacenter, vcenter, domain, customization_spec, notes: optional). docs/VM_DEPLOY_GUIDE.md<br>**Example:** `vm_deploy_ticket: CHG0001234`<br>*When:* the survey's one question - the operator types the change's number at launch; never set on the template |
| `vm_deploy_ticket_table`<br>Default: (empty)<br>**Leave as is** | leave it empty (the job finds each ticket's table); a table of your own, e.g. u_vm_request: for numbers whose prefix the job does not know (CTASK stays change_task)<br>**Example:** `vm_deploy_ticket_table: u_vm_request`<br>*When:* a table of your own whose number prefix the job does not know |
| `vm_deploy_ticket_prefixes`<br>Default: `{}`<br>**Leave as is** | Your instance's own number prefixes and their tables, e.g. {INCN: incident, CHGN: change_request}. Rarely needed: for a prefix it does not know, the job asks ServiceNow's task table (and takes INCN as INC, CHGN as CHG when it may not read that table). A table as people say it is understood: change / change request -> change_request, change task -> change_task, catalog task -> sc_task.<br>**Example:** `vm_deploy_ticket_prefixes: {CHGN: change_request, INCN: incident}`<br>*When:* your instance numbers tickets with its own prefixes and the API account may not read the task table |

| Setting | What it does - an example - when to set it |
|---|---|
| `vm_deploy_parent_tables`<br>Default: `[change_request]`<br>**Leave as is** | The request may also be on the ticket's parent - e.g. a task (SCTASK, CTASK) whose change holds the requirements: the parent's 'key: value' lines (and, with vm_deploy_plain_words, its plain words) are read too, under the ticket's own (what the ticket says wins). A parent that cannot be read, is closed, or is not ready (vm_deploy_ready_by_table) stops the build. Only a parent in one of these tables is read ([] = never read a parent):<br>**Example:** `vm_deploy_parent_tables: [change_request, sc_req_item]`<br>*When:* a catalog task whose requested item (RITM) holds the request - with request_item in vm_deploy_parent_fields |
| `vm_deploy_parent_fields`<br>Default: `[change_request, parent]`<br>**Leave as is** | The ticket's fields that point to its parent - the first one set is used: a change task's change_request, any ticket's parent (or your instance's own field, e.g. u_change):<br>**Example:** `vm_deploy_parent_fields: [change_request, request_item, parent]`<br>*When:* the field that points to the parent is another one (a catalog task's request_item, a field of your own) |
| `vm_deploy_parent_text_fields`<br>Default: `[description]`<br>**Set when needed** | The parent's fields with the request, in this order (a later one wins), e.g. [description, implementation_plan]; comments and work_notes may be listed too (oldest first). GenAI reads them as well.<br>**Example:** `vm_deploy_parent_text_fields: [description, implementation_plan, comments]`<br>*When:* launched with the task, and the change keeps the request in its implementation plan, or answers come as comments on it |
| `vm_deploy_parent_wins`<br>Default: `false`<br>**Set when needed** | What wins, from the least to the most: the parent (the change), the ticket's short description and description, its work notes, its comments (each oldest first) - a 'key: value' line and plain words alike: the later one wins, however each is written. So the change task wins over its change, and a comment wins over a work note. true = the parent's text wins over the ticket's short description and description instead (a change task that repeats the request, maybe wrongly); its work notes and comments still win over the parent.<br>**Example:** `vm_deploy_parent_wins: true`<br>*When:* launched with the task, and the task repeats the change's request (maybe wrongly) - the change's description wins |
| `vm_deploy_notes_also`<br>Default: `[]`<br>**Standard setup** | Every note and comment this job writes on the ticket goes on these too (closing never does: only the ticket itself is closed): parent = the change a task belongs to (when it was read); tasks = when the job is launched with a change, its open change tasks assigned to vm_deploy_assignment_group (and matching vm_deploy_notes_tasks_query). e.g. [parent, tasks]: the change and its task both get them, whichever the job is launched with. A note ServiceNow refuses there is a warning, not a failure.<br>**Example:** `vm_deploy_notes_also: [parent, tasks]`<br>*When:* every note on the change and on its task - whichever of the two numbers you launch with |

| Setting | What it does - an example - when to set it |
|---|---|
| `vm_deploy_notes_tasks_query`<br>Default: (empty)<br>**Standard setup** | An extra ServiceNow query for the change's tasks that get the notes, e.g. "short_descriptionLIKEto deploy" ("" = the group's)<br>**Example:** `vm_deploy_notes_tasks_query: "short_descriptionLIKEto deploy"`<br>*When:* which of the change's tasks get the notes (and are closed), by their words - copy it from the change task list's filter |
| `vm_deploy_notes_tasks_limit`<br>Default: `50`<br>**Leave as is** | the change's tasks that get the notes (and are closed), at most - more is a warning<br>**Example:** `vm_deploy_notes_tasks_limit: 100`<br>*When:* a change with more than 50 open tasks that match - the job warns when it reaches the limit |
| `vm_deploy_close_tasks`<br>Default: `false`<br>**Standard setup** | Launched with a change: also close its tasks found that way (with vm_deploy_done_fields_by_table's change_task entry, when every VM is built) - the change itself is not closed by this. They get the notes too.<br>**Example:** `vm_deploy_close_tasks: true`<br>*When:* the one switch - launched with the change, close its task when every VM is built; false = your team closes it |
| `vm_deploy_ready_by_table`<br>Default: `change_request: {approval: Approved, state: Implement}`<br>**Leave as is** | Before anything is built, a ticket of these tables - the ticket itself, or the parent it belongs to (a change task's change) - must have these fields: one value or a list (any of them), written as the ticket's form shows them (upper / lower case ignored). Checked by the dry run, the propose step AND right before the build: a change sent back while the approval waited is not built. Your instance's labels, e.g. {change_request: {approval: Approved, state: [Implement, Scheduled]}}; {} = no check.<br>**Example:** `vm_deploy_ready_by_table: {change_request: {approval: Approved, state: [Implement, Scheduled]}}`<br>*When:* your change states have other labels, or a scheduled change may be built too |
| `vm_deploy_templates`<br>Default: `[]`<br>**Standard setup** | What a ticket may ask for (names; * and ? allowed). Templates MUST be listed: nothing else is deployed. The others: [] = any. e.g. [rhel9-gold, win2022-gold]<br>**Example:** `vm_deploy_templates: [rhel9-gold, WIN2022-TEMPLATE]`<br>*When:* always - only these templates are ever deployed (* and ? allowed) |
| `vm_deploy_clusters`<br>Default: `[]`<br>**Set when needed** | clusters (and standalone hosts) a ticket may use; also the candidates of automatic placement<br>**Example:** `vm_deploy_clusters: ["PROD-CL*"]`<br>*When:* limit the clusters a ticket may name - and let the job choose among them when nothing names one |
| `vm_deploy_datastores`<br>Default: `[]`<br>**Set when needed** | datastores (or datastore clusters) a ticket may use; also the candidates of automatic placement<br>**Example:** `vm_deploy_datastores: ["DSC-PROD*"]`<br>*When:* limit the datastores (or datastore clusters) - and let the job choose among them |

| Setting | What it does - an example - when to set it |
|---|---|
| `vm_deploy_folders`<br>Default: `[]`<br>**Leave as is** | VM folders a ticket may use (paths below the datacenter's VM folder, e.g. Linux/*)<br>**Example:** `vm_deploy_folders: ["Servers/*"]`<br>*When:* limit the VM folders a ticket may name |
| `vm_deploy_max_cpu`<br>Default: `16`<br>**Set when needed** | the most vCPUs a ticket may ask for<br>**Example:** `vm_deploy_max_cpu: 8`<br>*When:* a lower cap than 16 vCPUs per VM |
| `vm_deploy_max_memory_gb`<br>Default: `128`<br>**Set when needed** | the most memory a ticket may ask for, in GB<br>**Example:** `vm_deploy_max_memory_gb: 64`<br>*When:* a lower cap than 128 GB per VM |
| `vm_deploy_max_vms`<br>Default: `10`<br>**Leave as is** | A ticket may ask for several VMs ("2 VMs named web01 and web02", "vm_name: web01, web02", or "Hostnames:" and one name per line): one VM each, the same size - or each its own, from a line of its own - built one after the other, the ticket closed when all are built. More names than this: refused, nothing built (the limits above are per VM).<br>**Example:** `vm_deploy_max_vms: 4`<br>*When:* fewer VMs per ticket than 10 |
| `vm_deploy_name_regex`<br>Default: `'^[A-Za-z0-9][A-Za-z0-9-]{0,62}$'`<br>**Leave as is** | The VM names allowed (a regular expression); e.g. '^[a-z0-9-]{1,15}$' for Windows-safe names.<br>**Example:** `vm_deploy_name_regex: '^[A-Za-z]{2}[0-9]{2}[A-Za-z0-9-]{1,11}$'`<br>*When:* your naming standard - here two letters, two digits, then up to 11 more (15 in all, as Windows allows) |
| `vm_deploy_assignment_group`<br>Default: (empty)<br>**Standard setup** | Only tickets assigned to this group (its name as ServiceNow shows it), or to one of a list of groups (["Group A", "Group B"]); "" = any group (the pickup needs one).<br>**Example:** `vm_deploy_assignment_group: Server Team`<br>*When:* only tickets (and the change's tasks) of this group; "" = any group - the task then found by its words alone (the standard setup) |
| `vm_deploy_assignment_group_by_table`<br>Default: `{}`<br>**Set when needed** | The same for the ticket the job is launched with, by its table - in place of vm_deploy_assignment_group for that table; "" = any group. e.g. {change_request: ""}: launched with a change, the change may be assigned to any group (it must still be approved: vm_deploy_ready_by_table); its tasks that get the notes (vm_deploy_notes_also, vm_deploy_close_tasks) are still those of vm_deploy_assignment_group.<br>**Example:** `vm_deploy_assignment_group_by_table: {change_request: ""}`<br>*When:* with vm_deploy_assignment_group set - the change itself may be any group's, its task still your group's |

| Setting | What it does - an example - when to set it |
|---|---|
| `vm_deploy_defaults`<br>Default: `{}`<br>**Standard setup** | What a ticket does not say: e.g. {cluster: PROD-CL01, datastore: DS-CLUSTER-01, folder: Linux/New}. Not set = the template's own cluster, datastore and folder; its CPU and memory.<br>**Example:** `vm_deploy_defaults: {template: rhel9-gold, cpu: 2, memory_gb: 8, cluster: PROD-CL01, datastore: DSC-PROD, folder: Servers/New}`<br>*When:* what a ticket leaves out - template, size and place included; what the ticket says still wins |
| `vm_deploy_sites`<br>Default: `{}`<br>**Set when needed** | Sites: a ticket's 'site: A' line (or its Location field, or with plain words a site's name or its words in vm_deploy_plain_sites - each VM by its own line) picks the placement - over vm_deploy_defaults and the name rules; a cluster / datastore / folder the ticket names itself still wins. A site in another vCenter: its vcenter (in vmware_vcenters, the same account) - and the template there under the same name. vm_deploy_sites: SiteA: {cluster: A-PROD-CL01, datastore: A-DSC01, folder: Servers/New} SiteB: {cluster: B-PROD-CL01, datastore: B-DSC01, folder: Servers/New, vcenter: vc02.yoursite.mil}<br>**Example:** `vm_deploy_sites: {SiteA: {cluster: A-CL01, datastore: A-DS01, folder: Servers/New}, SiteB: {vcenter: vc02.example.mil, cluster: B-CL01, datastore: B-DS01, folder: Servers/New}}`<br>*When:* several places to build - the ticket's site line or its Location picks one |
| `vm_deploy_site_required`<br>Default: `true`<br>**Leave as is** | with sites set up: a ticket must say its site (false = vm_deploy_defaults); a name rule counts too<br>**Example:** `vm_deploy_site_required: false`<br>*When:* with sites set up - a ticket that names no site gets vm_deploy_defaults instead of a refusal |
| `vm_deploy_name_rules`<br>Default: `[]`<br>**Set when needed** | Name rules: the VM's name picks where it goes - the first rule whose 'match' fits the name (* and ?, upper / lower case alike) fills in what the ticket does not say: a cluster (and datastores, folder ...) or a site. The ticket's own lines, and a site the ticket names, still win. A name used at several sites: 'sites' - the ticket must say which (its Location), never a guess; its placement is in vm_deploy_sites. A rule's placement is never used for a VM its site puts in another vCenter. vm_deploy_name_rules: - {match: "*AB01*", cluster: CLUSTER-ONE} # every name with AB01 in it - {match: "*CD02*", cluster: CLUSTER-TWO, folder: Servers/New} - {match: "*EF03*", site: SiteB} # or a site from vm_deploy_sites - {match: "*GH04*", sites: [SiteA, SiteB]} # at either site: the ticket says which [{match: "*AB01*", cluster: ...}, ...] - see above<br>**Example:** `vm_deploy_name_rules: [{match: "*AB01*", site: SiteA}, {match: "*XY05*", cluster: B-CL01}]`<br>*When:* the VM's name says where it goes (a site code in the name) |
| `vm_deploy_pxe_templates`<br>Default: `[]`<br>**Set when needed** | Templates with no operating system yet, that install it themselves: they boot from the network (PXE, e.g. an MECM task sequence) or from their CD / ISO (e.g. a kickstart). No guest customization, no waiting for an IP address; the MAC address and the computer name go on the ticket. e.g. [win2022-pxe]<br>**Example:** `vm_deploy_pxe_templates: ["WIN20*-PXE"]`<br>*When:* empty templates that install their OS from the network (MECM) or a CD - no customization, the MAC on the ticket |

| Setting | What it does - an example - when to set it |
|---|---|
| `vm_deploy_guestinfo`<br>Default: `true`<br>**Leave as is** | Give a VM from one of those empty templates the ticket's name and number as VM settings its installer can read: guestinfo.hostname and guestinfo.ticket (a kickstart with VMware Tools in the installer: vmtoolsd --cmd "info-get guestinfo.hostname"). Needs the vCenter privilege Change configuration > Advanced configuration; without it the VM is still built (a note says so). false = none.<br>**Example:** `vm_deploy_guestinfo: false`<br>*When:* the vCenter account lacks Advanced configuration, and the installer reads no guestinfo anyway |
| `vm_deploy_template_aliases`<br>Default: `{}`<br>**Leave as is** | Short names for long template names - for the ticket's 'template:' line and GenAI's catalog (whose keys may be these short names): {short name: the template's name in vCenter}, e.g. {win2022: "Windows 2022 Standard Template - Site 1", rhel9: "RHEL 9 Server Template"} A ticket may also write a template's own name in other case or spacing: it is matched to the spelling in vm_deploy_templates.<br>**Example:** `vm_deploy_template_aliases: {win2022: "Windows 2022 Standard Template - Site 1", rhel9: rhel9-gold}`<br>*When:* long template names - tickets and GenAI write the short one |
| `vm_deploy_template_specs`<br>Default: `{}`<br>**Standard setup** | The customization specification (vCenter > Policies and Profiles) for each template, when the ticket, its site and the name rules name none: {template name or pattern: spec}. A Windows template that is not network-boot needs one (Sysprep: administrator password, domain or workgroup), e.g. {"Windows 2022*": Win2022-Sysprep-Domain}. Linux templates need none (the job's own DHCP one).<br>**Example:** `vm_deploy_template_specs: {"WIN2022*": Win2022-Sysprep}`<br>*When:* every Windows template that is not network-boot - its Sysprep specification (password, domain) |
| `vm_deploy_max_disks`<br>Default: `5`<br>**Leave as is** | disks a ticket may ask for, disk 1 included (disks_gb: 100, 200 = disk 1 grown to 100 GB, one 200 GB disk added)<br>**Example:** `vm_deploy_max_disks: 3`<br>*When:* fewer disks per VM than 5 (disk 1 included) |
| `vm_deploy_max_disk_gb`<br>Default: `2048`<br>**Leave as is** | the biggest disk a ticket may ask for, in GB<br>**Example:** `vm_deploy_max_disk_gb: 1024`<br>*When:* a smaller biggest disk than 2 TB |
| `vm_deploy_max_total_disk_gb`<br>Default: `4096`<br>**Leave as is** | all its disks together, in GB<br>**Example:** `vm_deploy_max_total_disk_gb: 2048`<br>*When:* less disk per VM in all than 4 TB |
| `vm_deploy_mem_target_pct`<br>Default: `80`<br>**Leave as is** | Automatic placement - when the ticket, its site and vm_deploy_defaults name no cluster / datastore: the job chooses among a site's 'clusters' / 'datastores' lists (e.g. SiteA: {clusters: [A-PROD-*]}), else among vm_deploy_clusters / vm_deploy_datastores; with neither, the template's own. a cluster fits when its memory, this VM included and a host down, stays under this %<br>**Example:** `vm_deploy_mem_target_pct: 70`<br>*When:* automatic placement keeps more memory free (a cluster fits while it stays under this, a host down) |

| Setting | What it does - an example - when to set it |
|---|---|
| `vm_deploy_failover_hosts`<br>Default: `1`<br>**Leave as is** | hosts to plan as failed when choosing a cluster<br>**Example:** `vm_deploy_failover_hosts: 2`<br>*When:* plan for two hosts down when choosing a cluster (large clusters) |
| `vm_deploy_datastore_max_used_pct`<br>Default: `85`<br>**Leave as is** | a datastore fits when it stays under this % with the VM at its full size<br>**Example:** `vm_deploy_datastore_max_used_pct: 80`<br>*When:* automatic placement keeps more datastore space free |
| `vm_deploy_datastore_rules`<br>Default: `[]`<br>**Set when needed** | Datastores by the VM's name: the first rule whose match fits the name (* and ?, any case) gives the datastores it goes on - automatic placement chooses among them on its cluster, the least used first, over a site's or vm_deploy_defaults' datastore (one the ticket names must be one of them). A VM no rule matches never goes on a datastore a rule names: they are kept for those VMs. e.g. SQL servers: - {match: ["*SQL*", "*DBSE*", "*DBV*"], datastores: ["*SQL*"]}<br>**Example:** `vm_deploy_datastore_rules: [{match: ["*SQL*", "*DBSE*", "*DBV*"], datastores: ["*SQL*"]}]`<br>*When:* some servers have datastores of their own (SQL servers on the SQL datastores) - and no other VM goes there |
| `vm_deploy_datastore_balance`<br>Default: `false`<br>**Set when needed** | true = a VM whose ticket, site, name rule and vm_deploy_defaults name no datastore (and no vm_deploy_datastores) goes on the least used shared datastore of its cluster - the location's - the VMs of one ticket spread out (each counts the ones before it); false = the template's own datastore.<br>**Example:** `vm_deploy_datastore_balance: true`<br>*When:* spread the VMs - each on the least used shared datastore of its cluster, not the template's |
| `vm_deploy_resume`<br>Default: `true`<br>**Leave as is** | Run again for the same ticket (a job that stopped: failed, cancelled, timed out), pick up where it stopped: each VM carries its ticket in its notes from the clone on - one it built is left as it is, one it started and did not finish (cloned, not yet powered on) is finished where it is; the work notes say which. false = a VM with the name is refused, as any other.<br>**Example:** `vm_deploy_resume: false`<br>*When:* a job that stopped must never finish a VM it started - a VM of that name is refused, delete it by hand |
| `vm_deploy_domain`<br>Default: `localdomain`<br>**Set when needed** | the DNS domain Linux customization sets (a ticket's domain: wins)<br>**Example:** `vm_deploy_domain: example.mil`<br>*When:* the DNS domain Linux VMs get from guest customization |
| `vm_deploy_customize`<br>Default: `true`<br>**Leave as is** | guest customization: host name = vm_name, every adapter on DHCP<br>**Example:** `vm_deploy_customize: false`<br>*When:* the template sets itself up (cloud-init) - vCenter sets no host name or DHCP |
| `vm_deploy_power_on`<br>Default: `true`<br>**Leave as is** | power the new VM on (false = leave it off, e.g. to check it first)<br>**Example:** `vm_deploy_power_on: false`<br>*When:* someone checks the VM before it is switched on |

| Setting | What it does - an example - when to set it |
|---|---|
| `vm_deploy_wait_customization`<br>Default: `900`<br>**Leave as is** | seconds to wait for vCenter's customization result<br>**Example:** `vm_deploy_wait_customization: 1800`<br>*When:* a slow Sysprep (Windows joining the domain) |
| `vm_deploy_wait_ip`<br>Default: `600`<br>**Leave as is** | seconds to wait for an IPv4 address (DHCP, through VMware Tools)<br>**Example:** `vm_deploy_wait_ip: 1200`<br>*When:* slow DHCP on the staging network |
| `vm_deploy_done_fields`<br>Default: `{}`<br>**Leave as is** | Fields set on the ticket when the VM is up, e.g. to resolve an incident: {state: "6", close_code: "Solution provided", close_notes: "VM deployed"} (your instance's values)<br>**Example:** `vm_deploy_done_fields: {state: "6", close_code: "Solution provided", close_notes: "VM deployed"}`<br>*When:* NEVER with changes - it closes every ticket without its own entry, a change too; for incidents prefer vm_deploy_done_fields_by_table {incident: ...} |
| `vm_deploy_done_fields_by_table`<br>Default: `{}`<br>**Standard setup** | The same for one table (the ticket's table: incident, sc_task, change_task ...), in place of vm_deploy_done_fields for it - e.g. close the task, never the change it belongs to: {sc_task: {state: "3", close_notes: "VM deployed by AAP"}} (3 = Closed Complete; your instance's values) Sent on its own, after the result note: when ServiceNow refuses it (a mandatory field, a business rule) the note stays, and the job says so on the ticket and in its report (it stays green: the VM is built).<br>**Example:** `vm_deploy_done_fields_by_table: {change_task: {state: "3", close_code: successful, close_notes: "VM deployed by AAP"}}`<br>*When:* how a task is closed (your instance's values) - launched with the change, used only with vm_deploy_close_tasks |
| `vm_deploy_done_fields_issues_by_table`<br>Default: `{}`<br>**Standard setup** | Over those fields when every VM is built but something did not finish: no IPv4 address in time, no customization result, the guest's host name not applied, no network at power on, guestinfo not set - e.g. {change_task: {close_code: successful_issues}} (your instance's values; {} = closed the same way). What happened is on the ticket's work notes.<br>**Example:** `vm_deploy_done_fields_issues_by_table: {change_task: {close_code: successful_issues}}`<br>*When:* another close code when the VMs are built but something did not finish (no IP address in time ...) |
| `vm_deploy_done_fields_pxe`<br>Default: `true`<br>**Set when needed** | Also for a VM from a network-boot template (vm_deploy_pxe_templates), which is powered on and still installing its OS from MECM when the job ends. false = that ticket stays open for whoever checks the install.<br>**Example:** `vm_deploy_done_fields_pxe: false`<br>*When:* a network-boot VM's task stays open until someone checks the MECM install |

## The server's service, picked at launch: words in the VM's notes, an AAP inventory and group

| Setting | What it does - an example - when to set it |
|---|---|
| `vm_deploy_service`<br>Default: (empty)<br>**Set when needed** | What the new server is for, picked by whoever launches the deploy: a survey question (multiple choice, NOT required - the scheduled pickup starts the workflow without it), e.g. sql or dc. Every VM of the ticket gets it. "" = none. Its words go in each VM's notes in vCenter; with an inventory in vm_deploy_services, each VM built joins that AAP inventory and group.<br>**Example:** `vm_deploy_service: sql`<br>*When:* a survey question (multiple choice, not required) whose choices are the names in vm_deploy_services; never set on the template |
| `vm_deploy_services`<br>Default: `{}`<br>**Set when needed** | The services the survey offers: each one's AAP inventory and group, and the words for the VM's notes (note: left out = "Service: <its name>."). {} = any service is only written in the notes. vm_deploy_services: sql: {inventory: Windows servers, group: win_sql, note: SQL Server} dc: {inventory: Windows servers, group: win_dc, note: Domain controller} web: {inventory: Linux servers, group: web} test: {note: Test server - delete after 30 days} # the notes only A service with an inventory needs a "Red Hat Ansible Automation Platform" credential on the deploy template, whose AAP user has the Admin role on that inventory. The inventory and the group must exist - the job never creates them, it refuses (before anything is built) and says what to create. A service that is not listed is refused the same way: the survey's choices must be these names.<br>**Example:** `vm_deploy_services: {sql: {inventory: Windows servers, group: win_sql, note: SQL Server}, web: {inventory: Linux servers, group: web}}`<br>*When:* each new server should join its AAP inventory and group, and say what it is in its vCenter notes |
| `vm_deploy_inventory_name`<br>Default: `"{name}"`<br>**Set when needed** | The name a VM gets in the inventory: write it the way your inventory already names servers, or the same server shows up twice in AAP. {name} = the VM's name, e.g. "{name}.yoursite.mil". A service's own name: wins (e.g. {inventory: Windows servers, name: "{name}.corp.yoursite.mil"}).<br>**Example:** `vm_deploy_inventory_name: "{name}.yoursite.mil"`<br>*When:* your inventory names servers by their full DNS name - write it the same way, or a server shows up twice |
| `vm_deploy_inventory_lowercase`<br>Default: `true`<br>**Leave as is** | the inventory name in lower case (APP01 -> app01); false = as the ticket writes it<br>**Example:** `vm_deploy_inventory_lowercase: false`<br>*When:* your inventory keeps the names in capitals, as the ticket writes them |
| `vm_deploy_aap_ca_path`<br>Default: (empty)<br>**Leave as is** | a CA file for AAP's own HTTPS certificate ("" = vm_deploy_poll_aap_ca_path, else what the execution environment trusts)<br>**Example:** `vm_deploy_aap_ca_path: /etc/pki/ca-trust/source/anchors/yoursite-ca.pem`<br>*When:* AAP's own HTTPS certificate is from a CA the execution environment does not trust (and the pickup's vm_deploy_poll_aap_ca_path is not set) |

## A request in plain words, without GenAI (off by default)

| Setting | What it does - an example - when to set it |
|---|---|
| `vm_deploy_plain_words`<br>Default: `false`<br>**Set when needed** | true = also read the ticket's own words, by the rules below, so a requester can write "a Red Hat 9 server named ab01web05, medium" instead of 'key: value' lines. Never a guess: two different answers in the text are a question to the requester (a comment). 'key: value' lines still work and win; with vm_deploy_genai also on, GenAI is asked when the words leave the name or template - or a question - open.<br>**Example:** `vm_deploy_plain_words: true`<br>*When:* requesters write sentences, or one line per server, instead of 'key: value' lines |
| `vm_deploy_plain_templates`<br>Default: `{}`<br>**Set when needed** | The words for each template (any case; "windows 2022" also matches "Windows Server 2022", "server 2025" also "a new 2025 VM"). An allowed template's own name and its short names (vm_deploy_template_aliases) count too. An OS no template is for (Server 2025, only a 2022 template) is a question - never another version; to build it from a template whose name says another (an empty MECM shell): {"server 2025": <it>}. e.g. {"rhel 9": rhel9-gold, "red hat 9": rhel9-gold, "windows 2022": WIN2022-TEMPLATE}<br>**Example:** `vm_deploy_plain_templates: {"red hat 9": rhel9-gold, "windows 2022": WIN2022-TEMPLATE, "server 2025": WIN2025-PXE}`<br>*When:* with plain words - the words for each template |
| `vm_deploy_plain_sizes`<br>Default: `{}`<br>**Set when needed** | Size words and what they mean, e.g. {small: {cpu: 2, memory_gb: 8}, medium: {cpu: 4, memory_gb: 16}, large: {cpu: 8, memory_gb: 32}}. "4 vCPU" and "16 GB RAM" in the ticket are read too, and win in the same text.<br>**Example:** `vm_deploy_plain_sizes: {small: {cpu: 2, memory_gb: 8}, medium: {cpu: 4, memory_gb: 16}, large: {cpu: 8, memory_gb: 32}}`<br>*When:* with plain words - "a medium server" means a size |
| `vm_deploy_plain_sites`<br>Default: `{}`<br>**Set when needed** | The words for each site (vm_deploy_sites), as people write the place - any case, the spaces optional ("Site Two" is also "SiteTwo"). A site's own name counts too. A change that lists its servers one per line ('new VM / ab01web05 / ... Location: Site Two') puts each VM at the site its own line names; the CPU, memory and disks on each line are that VM's too. e.g. {"site two": SiteB, "north campus": SiteA}<br>**Example:** `vm_deploy_plain_sites: {"north campus": SiteA, "south campus": SiteB}`<br>*When:* with sites - the ways people write each place (its Location) |

## GenAI reads a request written in plain words (a proof of concept, off by default)

| Setting | What it does - an example - when to set it |
|---|---|
| `vm_deploy_genai`<br>Default: `false`<br>**Leave as is** | When the ticket's own 'key: value' lines do not give vm_name and template: ACT (GenAI, evidence mode, no commands) copies out what the ticket says - the names, the operating system as written, CPU, memory, disks - each with the ticket's words it comes from. It decides and asks nothing (its own questions are notes in the report): the job finds the template from the OS words by its own rules, and asks the requester (a comment) only what it needs and cannot read. That run builds NOTHING: it posts the proposal on the ticket. The next run builds that proposal (GenAI is not asked again), while the ticket text is unchanged; the ticket's own lines still win. Needs the ACT model key credential on the template. docs/VM_DEPLOY_GUIDE.md<br>**Example:** `vm_deploy_genai: true`<br>*When:* plain-words tickets the rules cannot read - needs the ACT model key credential |
| `vm_deploy_genai_in_check`<br>Default: `false`<br>**Leave as is** | A dry run (Check) asks GenAI too (true): it shows GenAI's proposal and where the VM would go, and writes nothing on the ticket - no proposal note, no questions to the requester. false = Check only says GenAI would be asked.<br>**Example:** `vm_deploy_genai_in_check: true`<br>*When:* a dry run shows GenAI's proposal too (and still writes nothing) |
| `vm_deploy_template_catalog`<br>Default: `{}`<br>**Leave as is** | Optional: templates GenAI may name by key, each with a plain description (the key: the template's exact name). Not needed when the templates' names say their OS: the job finds the template from the OS the ticket names. A key for another version than the ticket asks for is dropped. e.g. {win2022-pxe: "Windows Server 2022 (installed over the network by MECM)", rhel9-gold: "Red Hat Enterprise Linux 9"}<br>**Example:** `vm_deploy_template_catalog: {WIN2022-PXE: "Windows Server 2022 (installed over the network by MECM)"}`<br>*When:* GenAI on, and template names that do not say their OS |
| `vm_deploy_genai_approval`<br>Default: `true`<br>**Leave as is** | true: a GenAI proposal waits for a person - the workflow's approval (or a second run) - before it is built. false: one run reads, checks and builds, when every value is traced to the ticket and within the rules (questions or a refusal still stop it).<br>**Example:** `vm_deploy_genai_approval: false`<br>*When:* a demo with GenAI and no person in between - one run reads, checks and builds |
| `vm_deploy_phase`<br>Default: (empty)<br>**Leave as is** | Which step this run is: "" = one job does it all; propose = the workflow's first step (read, ask GenAI if needed, check, write where it would go on the ticket - build nothing); build = the step after the approval (build what the ticket and its proposal say - never ask GenAI).<br>**Example:** `vm_deploy_phase: propose`<br>*When:* set by the approval workflow's nodes (propose, then build) - never on the template |
| `vm_deploy_genai_model_timeout`<br>Default: `"{{ site_act_model_timeout \| default(600) }}"`<br>**Leave as is** | seconds the model may take for its answer<br>**Example:** `vm_deploy_genai_model_timeout: 900`<br>*When:* a slow model |

## Pick up VM request tickets on a schedule (playbooks/vm_deploy_poll.yml)

| Setting | What it does - an example - when to set it |
|---|---|
| `vm_deploy_poll_enabled`<br>Default: `true`<br>**Leave as is** | Every run (an AAP schedule, e.g. every 15 minutes) asks ServiceNow for NEW VM requests - active, assigned to vm_deploy_assignment_group (required), marked by vm_deploy_poll_prefix and/or vm_deploy_poll_query - and starts the deploy workflow once per ticket, after writing "[AAP] Picked up ..." on it. A ticket with any note of AAP's on it is never picked up again. Nothing is built by this job: the workflow reads, checks, asks for the approval and builds. Needs the "ServiceNow API" credential and a "Red Hat Ansible Automation Platform" credential (an AAP user that may only start that workflow). docs/VM_DEPLOY_GUIDE.md, "The scheduled pickup" false = the job runs, picks up nothing and says so (the off switch)<br>**Example:** `vm_deploy_poll_enabled: false`<br>*When:* the pickup's off switch - the schedule runs and picks up nothing |
| `vm_deploy_poll_tables`<br>Default: `[incident]`<br>**Set when needed** | the ServiceNow tables to look in: incident, sc_task, task ...<br>**Example:** `vm_deploy_poll_tables: [change_task]`<br>*When:* pick up change tasks rather than incidents |
| `vm_deploy_poll_prefix`<br>Default: `"VM request"`<br>**Set when needed** | a VM request's short description starts with this ("" = mark them by vm_deploy_poll_query)<br>**Example:** `vm_deploy_poll_prefix: ""`<br>*When:* the tickets are found by vm_deploy_poll_query, not by how their short description starts |
| `vm_deploy_poll_query`<br>Default: (empty)<br>**Set when needed** | An extra ServiceNow encoded query, ANDed with the rest, e.g. "category=vm_request" or "cmdb_ci.name=VM Request" ("" = none). ^NQ (a second query) is refused.<br>**Example:** `vm_deploy_poll_query: "change_request.approval=approved^change_request.state=-1"`<br>*When:* change tasks whose change is approved and in Implement (copy your own from the list's filter) |
| `vm_deploy_poll_states`<br>Default: `["1"]`<br>**Leave as is** | the ticket states to pick up (state values: "1" = New)<br>**Example:** `vm_deploy_poll_states: ["1", "-5"]`<br>*When:* pick up tasks in other states too (your instance's values; -5 = Pending out of the box) |
| `vm_deploy_poll_max`<br>Default: `3`<br>**Leave as is** | the most tickets started per run; the rest wait for the next run<br>**Example:** `vm_deploy_poll_max: 5`<br>*When:* more tickets started per run |
| `vm_deploy_poll_scan`<br>Default: `100`<br>**Leave as is** | the most tickets read per table and run, oldest first (tickets AAP already has count too)<br>**Example:** `vm_deploy_poll_scan: 200`<br>*When:* many tickets in the queue |
| `vm_deploy_poll_workflow`<br>Default: `"VM - deploy from ServiceNow ticket (workflow)"`<br>**Leave as is** | the workflow job template started per ticket<br>**Example:** `vm_deploy_poll_workflow: "VM - deploy (workflow, test)"`<br>*When:* your deploy workflow has another name |

| Setting | What it does - an example - when to set it |
|---|---|
| `vm_deploy_poll_dry_run`<br>Default: `false`<br>**Set when needed** | true = only list what it would pick up: no note, no workflow (Check does the same)<br>**Example:** `vm_deploy_poll_dry_run: true`<br>*When:* the first week - only list what it would pick up |
| `vm_deploy_poll_claim_fields`<br>Default: `{}`<br>**Set when needed** | Fields set on the ticket together with the "Picked up" note, e.g. {state: "2"} (In Progress) - so the ticket leaves the states picked up as well ({} = only the note).<br>**Example:** `vm_deploy_poll_claim_fields: {state: "2"}`<br>*When:* move a ticket picked up to In Progress (your instance's value) |
| `vm_deploy_poll_aap_ca_path`<br>Default: (empty)<br>**Leave as is** | a CA file for AAP's own HTTPS certificate ("" = what the execution environment trusts)<br>**Example:** `vm_deploy_poll_aap_ca_path: /etc/pki/ca-trust/source/anchors/site-ca.pem`<br>*When:* AAP's HTTPS certificate is from your own CA, which the execution environment does not trust |